

SLIDE: Shuffle Shamir Secret Shares Uniformly with Linear Online Communication and Guaranteed Output Delivery

Jiacheng Gao

Nanjing University
Nanjing, Jiangsu, China
jcgao@smail.nju.edu.cn

Yuan Zhang

Nanjing University
Nanjing, Jiangsu, China
zhangyuan@nju.edu.cn

Moyang Xie

Nanjing University
Nanjing, Jiangsu, China
xie_moyang@foxmail.com

Sheng Zhong

Nanjing University
Nanjing, Jiangsu, China
zhongsheng@nju.edu.cn

Abstract

We revisit shuffle protocols for Shamir secret sharing. Existing constructions either produce a non-uniform shuffle or incur high communication and round complexity, in some cases exponential in the number of parties. We propose two new shuffle protocols that achieve uniform shuffle with communication complexity $O\left(\frac{k+l}{\log k} n^2 m \log m\right)$ for shuffling rows of an $m \times l$ matrix shared among n parties, where $k \leq m$ is a tunable parameter that balances communication and computation. The first protocol is concretely more efficient, while the second achieves the best-known $O(nml)$ online communication and $O(n)$ rounds. Both constructions enjoy an ideal property of guaranteed output delivery (GOD). In terms of both overall and online communication, our protocols improve upon the state of the art for shuffle protocols based on Shamir secret sharing.

Our key technical ingredient is a novel permutation sharing technique that represents a permutation by smaller permutation matrices. Once permutations are shared, applying them becomes significantly cheaper, enabling a highly efficient online phase. The first protocol applies independent secret permutations from all parties in sequence, while the second protocol builds on the shuffle correlation technique of Gao et al., achieving $O(nml)$ online complexity. We further extend shuffle correlation to support GOD with linear online communication, an extension has not been previously explored. As a result, we obtain SLIDE, the first shuffle protocol achieving both $O(nml)$ online communication and guaranteed output delivery.

Our constructions rely only on the most basic primitives of Shamir secret sharing over any field $\mathbb{F}$ with $|\mathbb{F}| > n$. Since shuffle is a fundamental building block for higher-level MPC primitives such as sorting and oblivious data structures, our results are broadly applicable and can accelerate many real-world applications of secure multiparty computation.

CCS Concepts

• Security and privacy → Cryptography.

Keywords

Secure Multiparty Computation; Secure Shuffle; Shamir Secret Sharing

1 Introduction

Secure multiparty computation (MPC) has found wide applicability in real-world scenarios. In an MPC setting, multiple parties jointly evaluate a function over their private inputs. The goal is to preserve the confidentiality of each party's data while efficiently obtaining the correct output. This strong security guarantee makes MPC an attractive solution for many privacy-sensitive collaborative tasks.

Most MPC protocols are designed modularly, where existing primitives serve as subroutines to build new constructions. This modularity enables succinct protocol descriptions and clean security arguments via the composition theorem [11]. Among these primitives, the MPC shuffle protocol is particularly fundamental. An MPC shuffle protocol takes an array of secret-shared values as input from the parties holding them, randomly permutes it, and shares the result back to the parties such that no party knows which entry of the output comes from which entry of the input. Such an operation has numerous real-world applications, including constructing anonymous communication systems [4, 16, 33], electronic voting [25, 27, 38], distributed databases [41, 42], etc. Shuffle is also a versatile building block for more complex MPC primitives, including MPC sorting [10, 21, 22, 29], MPC oblivious RAM (ORAM) [17, 26, 37], etc.

Despite the broad application of MPC shuffles, recent research has focused on improving the efficiency of their additive secret sharing variants. A major breakthrough was achieved by Chase et al. [12], who proposed a novel semi-honest two-party shuffle protocol with almost linear computation and communication. Follow-up studies [16, 18, 28, 40] extended this line of work to active security and the multi-party setting, achieving linear online communication and computation. As a result, shuffle protocols based on additive secret sharing are already highly efficient.

In contrast, shuffle protocols for Shamir secret sharing [39] remain far less developed. The most efficient known approaches rely on switching networks [33, 34] or sorting networks [35, 36], with $O(nml \log m)$ communication for n parties shuffling m vectors of length l . However, these constructions exhibit significant drawbacks: switching-network-based shuffles fail to generate uniform shuffles [33, 34], while sorting-network-based shuffles achieve uniformity only with some probability [35, 36]. These limitations complicate the security analysis of higher-level MPC protocols built upon them.

Currently, uniform shuffle in Shamir secret sharing is known only through the permute-in-turn paradigm [26, 31], which incurs substantially higher complexities. The construction of [31] requires $O(2^{n^{1.5}})$ rounds, rendering it practical only for very small numbers of parties. The construction of [26] requires $O(n^2 ml \log m)$ communication and $O(n \log m)$ rounds, which is still less efficient than existing non-uniform shuffle protocols. These approaches are further discussed in Section 2, with a comparison of existing constructions provided in Table 1.

Contributions. This paper develops efficient and uniform shuffle protocols for Shamir secret sharing, with unconditional security against malicious parties under honest majority. These protocols are further enhanced to achieve guaranteed output delivery (GOD).

First, we adopt the approach of permute-in-turn to guarantee the uniformity of shuffle. At the core of our contributions is a novel permutation sharing technique that enables the parties to share a permutation and apply it to secret-shared data. Our technique requires $O(\frac{nmk \log m}{\log k})$ communication to share an m -permutation and $O(\frac{nml \log m}{\log k})$ communication to apply the shared permutation to m vectors of length l — here, $1 \leq k \leq m$ is a tunable parameter balancing communication and computation. By letting each party share one secret uniformly random permutation in the offline phase and sequentially applying them in the online phase, we obtain our first shuffle protocol.

To further improve online efficiency, we leverage the shuffle correlation technique of Gao et al. [18]. Originally, the shuffle correlation of Gao et al., only achieves security-with-abort, i.e. the honest parties abort without obtaining the output if the adversary deviates from the protocol. We therefore investigate how to instantiate their construction using our permutation sharing protocol. We show that, with the Shamir secret sharing scheme of [19], the dispute control technique [6] and the authentication tag technique of [8], we can securely generate and apply shuffle correlation with guaranteed output delivery (GOD). This yields SLIDE, the first uniform shuffle protocol that achieves both GOD and the best-known online communication complexity of $O(nml)$ alongside $O(n)$ rounds.

Our contributions are summarized as follows.

- (1) We introduce a novel primitive for sharing a permutation and applying it to secret-shared vectors. Using this primitive, we construct a shuffle protocol with $O(\frac{(k+l)n^2 m \log m}{\log k})$ total communication and $O(\frac{n \log m}{\log k} + \log \log k)$ round, where k is a tunable parameter. This protocol outperforms all existing uniform shuffle protocols for Shamir secret sharing. Moreover, for small $n = o(\log m)$, setting $k = \omega(2^n)$ achieves $o(nml \log m)$ online communication complexity and $o(\log m)$ online round complexity, surpassing even switching- and sorting-network-based approaches, at the cost of higher offline communication. (Section 4)
- (2) We enhance our shuffle protocol with the shuffle correlation technique of Gao et al. [18]. This yields a second shuffle protocol with the best-known $O(nml)$ online communication complexity and $O(n)$ rounds. Its offline communication complexity, $O(\frac{(k+l)n^2 m \log m}{\log k})$, also improves upon the previous

best-known bound of $O(n^2 ml \log m)$, under the uniformity requirement. (Section 5)

- (3) We enhance both our protocols and the shuffle correlation of Gao et al. [18] to achieve GOD, ensuring honest parties obtain the correct output regardless of adversarial behavior. This yields SLIDE, the first shuffle protocol that simultaneously achieves uniformity, GOD, and best-known $O(nml)$ online communication. We note that whether shuffle correlation could be extended to GOD was not previously known, and our work establishes this as a new result. (Section 6)
- (4) We analyze the communication, computation, and round complexities of our protocols to quantify the effect of k and guide the choice of its optimal value.

Organization The rest of the paper is organized as follows. Section 2 reviews prior MPC shuffle protocols. Section 3 introduces basic notations and security models used herein. Section 4 presents our permutation sharing technique and its derived shuffle protocol. Section 5 demonstrates leveraging this technique to construct a linear online complexity shuffle protocol, matching Section 4's overall complexity up to constants. Section 6 enhances our protocols to support guaranteed output delivery (GOD), including adapting Gao et al. [18]'s shuffle correlation. Due to space constraints, security proofs, complexity analysis proofs, and auxiliary discussions are deferred to Appendices A, B, and C, respectively. The rest of the paper is organized as follows. In Section 2, we review prior work on MPC shuffle protocols, with a focus on constructions for Shamir secret sharing. Section 3 introduces the basic notations and security models used throughout the paper. In Section 4, we present our permutation sharing technique and show how to build a shuffle protocol from it. Section 5 demonstrates how to leverage this technique to construct a shuffle protocol with linear online complexity, which matches the overall complexity of the protocol in Section 4 up to constants. Section 6 shows how to enhance our protocols to support guaranteed output delivery (GOD), including adapting the shuffle correlation of Gao et al. [18].

Due to space limitations, security proofs are provided in Appendix A, the proofs of our complexity analysis are given in Appendix B, and several auxiliary discussions are deferred to Appendix C.

2 Related Works

Secure MPC Shuffle. The concept of a multi-party shuffle originates from Chaum [14] as a byproduct of the decryption shuffle. In this scenario, clients iteratively encrypt messages using servers' public keys and send them to the first server. Each server then decrypts, randomly permutes, and forwards the message batch. After the final server processes the batch, the messages are securely shuffled, ensuring no server can link messages to their senders. While the original protocol assumes honest servers, subsequent works achieve active security [1, 2, 20, 24] through zero-knowledge proofs that verify the correctness of decryption and permutation. These approaches incur $O(m^2)$ computation per server and are generally heavy due to the overhead of public-key operations and proof generation.

Despite its complexity, the decryption shuffle technique is not suited for multi-party computation (MPC) tasks. After decryption

and permutation, its output consists of exactly shuffled plaintexts. In contrast, MPC scenarios require the output to remain secret-shared to enable subsequent secure computations on the data. For instance, in [21] and [10], the authors construct MPC sorting protocols based on shuffle, where it is essential that the shuffle outputs remain secret-shared to enable further secure computations.

A recent breakthrough by Chase et al. [12] significantly improved the efficiency of two-party MPC shuffle, thereby enhancing the efficiency of multi-party MPC and its shuffle-based applications. Their key idea is a permutation decomposition technique that breaks a large m -permutation into $O(\frac{m \log m}{k \log k})$ small k -permutations. Each k -permutation is implemented using obliviously punctured matrices (OPM) with $O(kl + k \log k)$ communication and $O(k^2 l)$ local computation, yielding a two-party shuffle protocol with $O(1 + \frac{l}{\log k})m \log m$ communication. At its core, the protocol relies only on oblivious transfer extension (OTE) and pseudo-random generator (PRG), which are computationally lightweight.

Han et al. [23] and Pecený et al. [37] identify the randomness underlying [12] as permutation correlation. Han et al. [23] improves its concrete generation efficiency, while Pecený et al. [37] proposes a new method with $O(m \log l)$ communication for generating such randomness, under the learning parity with noise (LPN) assumption. Initially designed for the two-party, semi-honest setting, this line of work has since been extended to the multi-party setting [28], to achieve linear online communication [15, 16], and to support malicious security [40]. The latest advance by Gao et al. [18] achieves both malicious security and linear online communication in the multi-party setting.

However, all the above advances in MPC shuffle protocols are designed for additive secret sharing, and cannot be directly applied to Shamir secret sharing. The gap stems from fundamental differences between the two schemes. In Shamir sharing, shares must be points on a low-degree polynomial, which excludes additive-sharing techniques such as PRG-based re-randomization used in Chase et al. [12] to balance computation and communication. Consequently, shuffle protocols over Shamir sharing face significantly stricter design constraints and efficiency limitations.

Secure MPC Shuffle for Shamir Secret Sharing. To date, three main approaches are known for constructing MPC shuffle protocols in Shamir secret sharing. The first is switching networks, introduced by Lu et al. [33] and optimized by Mardi et al. [34]. In this approach, a shuffle is implemented via layers of switch gates: each gate randomly swaps two inputs based on a random bit, and each gate can be realized using MPC multiplication. Mardi et al. [34] achieve an best-known communication complexity of $O(nml \log m)$. However, a critical limitation is that the resulting permutation is not uniform, as some permutations occur with significantly higher probability than others (see Table 4 of [34]), weakening the overall security guarantee.

The second approach is based on sorting networks, initially proposed by Movahedi et al. [35, 36]. The idea is to tag each data item with a random value; sorting on these tags yields a uniform shuffle. This reduces the problem to implementing an MPC sorting protocol. A key drawback, however, is that MPC sorting protocols rely on heavy MPC primitives. In addition, the design of sorting networks (or equivalently, oblivious sorting algorithms) is difficult.

Existing networks are either too deep (e.g., Batcher’s odd–even merge sort and bitonic sort [5], both with depth $O(\log^2 m)$), have impractically large constants (e.g., the AKS network[3]), or only sort correctly with a probability (e.g., the LP network [32]). This also explains why many MPC sorting protocols adopt the shuffle-then-sort paradigm [22, 29], eliminating the need for an oblivious sorting algorithm. Using the LP network, Movahedi et al. [35, 36] achieve $O(nml \log m)$ communication and a permutation that is uniform with a probability of $1 - O(m^{-3})$. Although it leaves a small (albeit non-negligible) probability of a non-uniform shuffle, this guarantee is already sufficient for many applications.

The third approach is permute-in-turn, adopted by Laur et al. [31] and Keller and Scholl [26]. In [31], the strategy is to have multiple subsets of parties agree on independent uniform permutations, apply these permutations to the array, and then re-share the result with all parties. Each subset must be large enough to allow secret reconstruction, and sufficiently many subsets are required so that an adversary cannot corrupt all of them. Their analysis shows that approximately $O(2^n / \sqrt{n})$ subsets are needed, resulting in a communication complexity of $O(2^n n^{1.5} ml \log m)$. Keller and Scholl [26] improve this paradigm by incorporating a switching network. In their construction, each party sequentially applies a permutation by sharing $O(m \log m)$ control bits that determine the network’s switching operations. As all parties permute sequentially, the resulting shuffle is uniform. This reduces the communication cost to $O(n^2 ml \log m)$ within $O(n \log m)$ rounds. The primary advantage of the permute-in-turn paradigm is its guarantee of uniformity. However, both variants incur higher communication and round complexities compared to switching- or sorting-network-based approaches. The construction of [31] requires a round complexity exponential in n , while the one in [26] adds a factor of n in communication complexity.

Beyond the aforementioned advantages and drawbacks, a crucial limitation of existing works is their neglect of online complexity. Many MPC protocols adopt a two-phase design consisting of a data-independent offline phase and a data-dependent online phase. This phase separation enables most heavy operations to be precomputed offline, leaving the online phase lightweight so that parties can respond swiftly to real-world data as it arrives. In Appendix C.2, we provide a more detailed discussion of why online complexity is often more crucial than offline complexity.

However, nearly all existing shuffle protocols have only a trivial offline phase that prepares basic randomness (e.g., shared random values or bits). For instance, switching-network-based protocols require all input data to be available before feeding them into the network, meaning only random bits can be precomputed offline. Similarly, sorting-based approaches cannot proceed without actual inputs, as there is no data to sort. Even though the construction of Keller and Scholl [26] allows parties to share and verify control bits offline, its online phase still incurs $O(n^2 ml \log m)$ communication, matching its offline cost. As a result, the online workload remains heavy and is a bottleneck in large-scale MPC applications.

Shuffle with Guaranteed Output Delivery. A desirable property of shuffle protocols is guaranteed output delivery (GOD), which ensures that honest parties obtain the correct output even in the presence of malicious behavior. Although seemingly demanding,

Table 1: Summary of existing MPC shuffle protocols based on Shamir secret sharing. Here, n is the number of parties, m the number of items to be shuffled, l the bit-length of each item, and k a tuning parameter balancing communication, computation, and round complexity. B and C denote the costs of generating one shared random bit and one MPC comparison, respectively, determined by the underlying implementation. “Off/On Comm” stands for “offline/online communication”, “Off/On Round” for “offline/online round complexity”. The protocols of [26, 34–36] are generic and not limited to Shamir secret sharing, although their asymptotic costs may differ under other MPC frameworks.

Protocol	Uniform	On Comm	On Round	Off Comm	Off Round	Approach
[33]	No	$O(nml \log^2 m)$	$O(\log^2 m)$	$O(Bm \log^2 m)$	$O(1)$	Switching Network
[34]	No	$O(nml \log m)$	$O(\log m)$	$O(Bm \log m)$	$O(1)$	Switching Network
[35][36]	w.p. ¹	$O(nml \log m)$	$O(\log m)$	$O(Cm \log m)$	$O(1)$	Sorting Network
[31]	Yes	$O(2^n n^{1.5} ml \log m)$	$O(2^n / \sqrt{n})$	$O(1)$	$O(1)$	Permute-in-turn
[26]	Yes	$O(n^2 ml \log m)$	$O(n \log m)$	$O(n^2 m \log m)$	$O(1)$	Permute-in-turn
Shuffle1	Yes	$O(\frac{n^2 ml \log m}{\log k})$	$O(\frac{n \log m}{\log k})$	$O(\frac{kn^2 m \log m}{\log k})$	$O(\log \log k)$	Permute-in-turn
Shuffle2 and SLIDE ²	Yes	$O(nml)$	$O(n)$	$O(\frac{(k+l)n^2 m \log m}{\log k})$	$O(\frac{\log m}{\log k} + \log \log k)$	Permute-in-turn

¹ According to [35], this probability is $1 - O(m^{-3})$.

² When $n \geq \kappa$, we have that the offline communication complexity reduces to $O(\frac{k+l}{\log k} \kappa nm \log m)$, and the online communication complexity reduces to $O(\kappa ml)$.

most existing shuffle protocols for Shamir secret sharing already achieve GOD [26, 33–36]. This is because these protocols are typically built solely on MPC multiplication, and thus naturally inherit GOD when instantiated with a Shamir secret sharing scheme that supports multiplication with GOD (e.g., [19]).

In contrast, achieving GOD for uniform shuffle protocols with linear online complexity is significantly more challenging. Linear-online shuffles were first obtained for semi-honest additive secret sharing by [16] and later extended to malicious security by [18]. For efficiency, both works rely on highly customized MPC primitives that are not designed to provide GOD, and simply replacing the underlying secret sharing scheme does not suffice to guarantee GOD. Consequently, whether a uniform shuffle protocol can simultaneously achieve linear online complexity and GOD has remained open.

Summary. In this paper, we present uniform shuffle protocols for Shamir secret sharing that substantially improve online efficiency. We first introduce a permutation sharing technique that enables a non-trivial trade-off between the communication cost of sharing a permutation and that of applying it. Unlike the share translation protocol of [12], which uses PRG to reduce communication in additive secret sharing by trading communication for computation, our construction leverages a structural property of Shamir sharing: inner products of shared vectors of length l can be computed with only $O(n)$ communication and $O(nl)$ computation. This yields our first shuffle protocol with $O(\frac{n^2 ml \log m}{\log k})$ online communication and $O(\frac{n \log m}{\log k})$ online rounds. To further reduce online complexity, we leverage the shuffle correlation technique of Gao et al. [18], obtaining our second shuffle protocol with $O(nml)$ online communication and $O(n)$ online rounds. We then enhance the shuffle correlation technique and our second shuffle protocol to achieve guaranteed output delivery (GOD). This yields SLIDE, the first uniform shuffle protocol that achieves both GOD and the best-known communication complexity of $O(nml)$ alongside $O(n)$ rounds.

3 Preliminary

3.1 Basic Notation

We consider an MPC scenario with n parties $P_1, P_2, \dots, P_n$ engaging the computation. Let $\mathbb{F}$ be a finite field satisfying $|\mathbb{F}| > n$, and denote its prime subfield by $\mathbb{F}_p$. Let $\mathbb{K}$ be an extension of $\mathbb{F}$, such that $|\mathbb{K}| \geq 2^\kappa$ for statistical security parameter κ . Let $\{\alpha_i\}_{i=1}^n$ be n distinct non-zero elements of $\mathbb{F}$, which are public to all parties. Let $t < n/2$ denote the number of corrupted parties.

We use bold letters (e.g., $\mathbf{x}, \mathbf{y}, \mathbf{z}$) to denote vectors. Vectors are column vectors unless stated otherwise. We denote the i -th entry of vector $\mathbf{x}$ as x_i or $\mathbf{x}(i)$. The latter is useful when the vector itself has a subscript, e.g., vector $\mathbf{x}_1, \mathbf{x}_2$. Bold uppercase letters (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{C}$) denote matrices, and the element on i -th row and j -th column of $\mathbf{A}$ is denoted by $a_{i,j}$ or $\mathbf{A}(i, j)$.

Define $[m] = \{1, 2, \dots, m\}$ for any positive integer m . For a vector $\mathbf{x} = (x_i)_{i \in [m]}$ of length m and an m -permutation $\pi : [m] \rightarrow [m]$, applying π to $\mathbf{x}$ yields

$$\pi(\mathbf{x}) := (x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(m)}).$$

The permutation matrix of permutation π is defined as $\mathbf{P}_\pi := (r_{i,j})_{i,j \in [m]}$, where

$$p_{i,j} = \begin{cases} 1 & \text{if } j = \pi(i), \\ 0 & \text{otherwise.} \end{cases}$$

Note that i is the row index, and j is the column index, and the permutation satisfies

$$\pi(\mathbf{x}) = \mathbf{P}_\pi \cdot \mathbf{x}.$$

We denote the Shamir secret sharing of a value s among all parties by $\llbracket s \rrbracket$. This means there exists a degree- t polynomial $f \in \mathbb{F}[X]$ (where t is the number of corrupted parties) such that $f(0) = s$ and $f(\alpha_i)$ is known exclusively to P_i . A secret-shared vector $\llbracket \mathbf{v} \rrbracket$ or matrix $\llbracket \mathbf{A} \rrbracket$ indicates that every entry is individually secret-shared. For notational simplicity, we write a shared vector as:

$$\llbracket \mathbf{v} \rrbracket = (\llbracket v_1 \rrbracket, \llbracket v_2 \rrbracket, \dots, \llbracket v_m \rrbracket)^\top = \llbracket v_1, v_2, \dots, v_m \rrbracket^\top,$$

with the same convention applying to shared matrices.

When invoking a sub-protocol, three types of parameters may be involved. For instance:

$$\text{Prot}(a, \llbracket b \rrbracket, P_i : c),$$

which denotes invoking protocol Prot with a constant a , a secret-shared value $\llbracket b \rrbracket$, and a private input c from party P_i .

We assume the items to be shuffled are m field elements or vectors, where m is a power of two. The main reason for this assumption is to leverage a technique proposed by Chase et al. [12], which employs a Beneš network [9] for permutation decomposition. Originally, the Beneš network is designed only for permutation of a power-of-2 size. Since shuffling m -sized array can be readily used to shuffling array of smaller size $m' \leq m$ (as discussed in [18]), this assumption is without loss of generality.

3.2 Security Model

We consider a malicious adversary capable of corrupting at most $t < n/2$ parties. We assume the existence of secure pair-wise channels and a secure broadcast channel. For simplicity, we measure communication complexity in terms of the number of field elements transmitted, implicitly assuming $\log |\mathbb{K}| = O(1)$. Thus, broadcasting a field element either from $\mathbb{F}$ or $\mathbb{K}$ incurs $O(n)$ communication.

For clarity, our primary objective is to construct shuffle protocols that achieve security-with-abort, with unconditional security against static adversaries under an honest majority. A static adversary selects and fixes the set of parties to corrupt before the protocol's initiation. Unconditional security means our construction does not rely on any computational hardness assumptions, i.e., the adversary may be computationally unbounded. Specifically, as long as the number of corrupted parties is less than $n/2$, our construction remains statistically secure: if corrupted parties deviate from the protocol, honest parties will abort with overwhelming probability $p > 1 - C \cdot 2^{-\kappa}$, where C denotes the number of MPC operations (i.e., the size of the computation circuit). Such a strong guarantee is known to be achievable only in the honest-majority setting [7, 13].

In Section 6, we extend our constructions to guarantee output delivery (GOD). GOD is a stronger security guarantee stating that honest parties always obtain correct results regardless of adversarial behavior. This is achieved by replacing all security-with-abort MPC primitives with their GOD counterparts [19], applying the dispute control framework of [6] to identify and exclude corrupted parties, and using the authentication tag mechanism of [8] to recover a correct intermediate state for honest parties.

For space reasons, security proofs are deferred to Appendix A.

3.3 Primitives for Shamir Secret Sharing Scheme

Our constructions require only a high-level interface to Shamir secret sharing, which we formalize through an ideal MPC functionality F_{Shamir} supporting the following commands:

- $\llbracket s \rrbracket \leftarrow \Pi_{\text{input}}(P_i : s)$: share a secret from P_i to all parties.
- $s \leftarrow \Pi_{\text{open}}(\llbracket s \rrbracket)$: publicly open a secret s to all parties.
- $s \leftarrow \Pi_{\text{open}}(\llbracket s \rrbracket, P_i)$: open s to party P_i .
- $\llbracket s \rrbracket \leftarrow \Pi_{\text{add}}(\llbracket a \rrbracket, \llbracket b \rrbracket)$: return $s = a + b$. This protocol is done without communication.

- $\llbracket s \rrbracket \leftarrow \Pi_{\text{mul}}(\llbracket a \rrbracket, \llbracket b \rrbracket)$: return $s = ab$ (with rerandomization).
- $\llbracket s \rrbracket \leftarrow \Pi_{\text{inner}}(\llbracket a \rrbracket, \llbracket b \rrbracket)$: return inner product $s = \mathbf{a}^\top \cdot \mathbf{b}$ (with rerandomization).
- $\llbracket r \rrbracket \leftarrow \Pi_{\text{rand}}(\mathbb{J})$: uniformly draw $r \xleftarrow{\$} \mathbb{J}$ where $\mathbb{J}$ is either $\mathbb{F}_p$, $\mathbb{F}$ or $\mathbb{K}$.
- $\lambda \leftarrow \Pi_{\text{challenge}}(\mathbb{K})$: uniformly draw a public λ over designated field $\mathbb{K}$.

We assume that the above functionalities are ideal: any misbehavior leads to immediate abort. This differs from real-world implementations, where deviations are detected only after certain correctness checks.

Note that Π_{input} , Π_{open} , Π_{mul} , Π_{rand} and $\Pi_{\text{challenge}}$ can be implemented in $O(n)$ communication and $O(n)$ computation. The inner-product primitive Π_{inner} , requires $O(nm)$ computation but only $O(n)$ communication, when invoked on vectors of length m . In our later constructions, we will leverage this asymmetric to make trade-off between communication and computation, achieving an overall optimized complexity.

A concrete realization of these primitives with guaranteed output delivery (GOD) is provided by [19]. In practice, implementations of F_{Shamir} rely on batching to achieve amortized linear communication. This amortized complexity is fully compatible with our protocols since we do not require immediate correctness checks for each multiplication. We also note that due to efficiency considerations, the concrete implementation of $\Pi_{\text{challenge}}$ produces a distribution only “close to” uniform, incurring $O(n)$ communication overhead [19]. For the simplicity of analysis, we abstract away this detail and instead assume perfect uniformity. This abstraction does not affect the security of our protocol.

3.4 Generic Approach for GOD

We enhance our constructions to guarantee output delivery (GOD) by leveraging the primitives and framework of Goyal et al. [19], which provides a generic approach for realizing GOD in the honest-majority setting via Shamir secret sharing. Their framework builds on the dispute control technique of Beerliova-Trubiniova and Hirt [6], a strategy for identifying corrupted parties.

Dispute Control. The core idea of the dispute control technique is maintain two public sets:

- (1) Corr, containing parties already identified as corrupted.
- (2) Disp, containing the pairs of parties P_i, P_j currently in dispute (meaning at least one of P_i and P_j must be corrupted).

It follows that if a party is disputed with at least $(t + 1)$ parties, it must be corrupted. Additionally, any party identified as corrupted is automatically placed in dispute with all remaining parties. The primitives proposed by Goyal et al. [19] are designed such that all honest parties consistently agree on the state of these two sets, and innocent parties are never wrongfully added to Corr.

Parties proceed with the computation. Whenever a correctness check fails, the parties revert to the beginning of the current segment and update Corr or Disp. As every failure enlarges one of the two sets, and as the number of potential disputes is $O(n^2)$, the computation will be rolled back at most $O(n^2)$ times, which is sufficient to identify all corrupted parties.

Communication between disputed parties is restricted: they never send messages directly to each other. If P_i needs to share a secret, it simply sets the share intended for P_j to zero. If P_i must send a non-sharing message to P_j , it routes the message through a relay $P_{i \leftrightarrow j}$, who is the lowest-index party not in dispute with either of them. Subsequently, any inconsistency in the relayed message forces this relay to side with one party, creating a new dispute and ensuring eventual corruption identification.

When a corrupted party is finally excluded from computation, its shares become unusable. This poses a challenge for multiplication computations, which needs at least $2t + 1$ valid shares to proceed. Goyal et al. [19] proposes a primitive dubbed “small surgery” to enable computation to continue by nullifying the shares held by corrupted parties. Note that this does not impact our constructions, as our constructions treat Π_{mul} and related primitives as black boxes. Hence, these details are abstracted away by our ideal-functional assumption.

Circuit Division. For efficiency optimization, the arithmetic circuit C is divided into $O(n^2)$ segments. Following the computation of each segment, a correctness check is performed to verify that all computations up to that point are correct. If the check passes, parties additionally compute authentication tags [8], which act as commitments to the shares produced in that segment. Broadly speaking, the purpose of these authentication tags is to prevent corrupted parties from lying about its shares obtained in earlier segments. The rationale is that if a party falsifies its shares for the current segment, parties can simply recompute that segment. However, if a party falsifies shares from a previous segment, rewinding the computation to that segment is impractical due to efficiency constraints. In such cases, the authentication tags will help identify the lying party and add it to Corr. This ensures the computation never reverts to previous segments. Since there will be at most $O(n^2)$ rollbacks and each segment having size $O(|C|/n^2)$, the overall computation remains $O(|C|)$, linear in the size of the circuit.

4 Permutation Sharing Protocol and Shuffle Protocol

In this section, we begin with constructing a permutation sharing protocol that enables a designated party P_w to efficiently share a secret permutation π , which can subsequently be applied to any vector of compatible size. Given such a permutation-sharing mechanism, a shuffle protocol follows directly via the standard permute-in-turn paradigm: in the offline phase, each party shares a secret uniform m -permutation; in the online phase, all parties apply all shared permutations sequentially. The uniformity of the final shuffle is guaranteed by the fact that even if only one permutation remains secret and uniform, the composite result is uniform.

Consider permuting m secret-shared vectors of length l . Two straightforward approaches for implementing such a permutation sharing scheme exist:

- (1) Share an $m \times m$ permutation matrix among parties. Sharing such a permutation incurs $O(nm^2)$ communication, while applying it incurs $O(nml)$ communication, $O(nm^2l)$ computation and $O(1)$ rounds.

- (2) Share $m \log m$ bits among parties. Each bit corresponds to a control bit in the permutation network [26]. Sharing a permutation hence incurs $O(nm \log m)$ communication, while applying it incurs $O(nml \log m)$ communication, $O(nml \log m)$ computation and $O(\log m)$ rounds.

The first approach achieves lower communication overhead for permutation application, while the second offers better overall communication complexity. The first approach’s key limitation is its $O(nm^2)$ communication and computation overhead for permutation sharing, prohibitive for large m . The second approach’s key drawback are its $O(\log m)$ round complexity, and the fact that its permutation application is no more efficient than its permutation sharing, meaning the protocol does not leverage the offline phase effectively.

Our construction unifies these extremes. We observe that an m -permutation can be represented by $\frac{m \log m}{k \log k}$ many $k \times k$ permutation matrices. Permutation application then reduces to multiplying these $k \times k$ matrices with k -dimensional vectors in the correct order. Sharing a permutation via our approach thus incurs $O(\frac{knm \log m}{\log k})$ communication, while application of the permutation incurs $O(\frac{nml \log m}{\log k})$ communication and $O(\frac{nmkl \log m}{\log k})$ computation.

To realize this, we require a protocol for sharing well-formed permutation matrices. However, verifying whether an arbitrarily shared matrix is a well-formed permutation matrix is generally non-trivial (see discussion in Appendix C.1). To address this challenge, we first enable a party to share k distinct one-hot vectors. By verifying that these k one-hot vectors are pairwise distinct, parties can confirm the well-formedness of the resulting permutation matrix.

We proceed as follows: we first formalize the definition of a shared permutation and illustrate how parties leverage it to execute permutation operations; we next present our construction for sharing one-hot vectors as a key building block for generating shared permutation matrices; subsequently, we detail our permutation sharing protocol construction; finally, we describe our shuffle protocol built upon our permutation sharing protocol, followed by a comprehensive complexity analysis.

4.1 Shared Permutation

Definition 4.1 (k -Shared m -Permutation). We say an m -permutation π is k -shared as $\llbracket \pi \rrbracket_k$, if

$$\llbracket \pi \rrbracket_k := (k, s, \{\{\llbracket \mathbf{P}_{\pi'_{i,j}} \rrbracket, T_{i,j}\}_{j \in [m/k]}\}_{i \in [s]}),$$

where

- (1) Each $\pi'_{i,j}$ is a k -permutation.
- (2) $s = O(\frac{\log m}{\log k})$. This bound is tight, c.f. Appendix C.3.
- (3) $\mathbf{P}_{\pi'_{i,j}}$ denotes permutation matrix corresponding to $\pi'_{i,j}$, which is secret-shared among parties.
- (4) $T_{i,j} \subseteq [m]$ is a subset of size k , referred to the “task” of $\pi'_{i,j}$. For a fixed i , the sets $T_{i,j}$ are disjoint.
- (5) $\pi = \pi_{s,1} \circ \pi_{s,2} \circ \dots \circ \pi_{s,m/k} \circ \pi_{s-1,1} \circ \dots \circ \pi_{1,m/k}$, where $\circ$ denotes permutation composition. Here, $\pi_{i,j}$ is the m -permutation induced by applying $\pi'_{i,j}$ to the entries specified by $T_{i,j}$.

For simplicity, we further require that m and k are power-of-2. When k is irrelevant to the context, we abbreviate $\llbracket \pi \rrbracket_k$ as $\llbracket \pi \rrbracket$.

Once $\llbracket \pi \rrbracket$ is shared, parties can apply π to any number of m -dimensional vectors. This is done by iteratively substituting the relevant entries of the m -vector with the product of the corresponding shared $k \times k$ permutation matrix and the extracted subvector. For instance, when $k = m$, the shared permutation reduces to a single secret-shared $m \times m$ permutation matrix, and multiplying it by the vector directly yields the permuted vector. This process is formalized in Algorithm 1.

Algorithm 1 $\llbracket \pi(v) \rrbracket \leftarrow \text{Permute}(\llbracket \pi \rrbracket, \llbracket v \rrbracket)$

Require: π is a shared m -permutation and v is of length m .

Ensure: Return $\llbracket \pi(v) \rrbracket$.

```

Parse  $\llbracket \pi \rrbracket$  as  $(k, s, \{\{\llbracket P_{\pi'_{i,j}} \rrbracket, T_{i,j}\}_{j \in [m/k]}\}_{i \in [s]})$ 
for  $i = 1$  to  $s$  do
  for  $j = 1$  to  $m/k$  do parallel
     $\llbracket u \rrbracket \leftarrow \llbracket v(l) \rrbracket_{l \in T_{i,j}}$ 
     $\llbracket u' \rrbracket \leftarrow \llbracket P_{\pi'_{i,j}} \rrbracket \cdot \llbracket u \rrbracket$  ▷  $k$  invocations of  $\Pi_{\text{inner}}$ .
     $\llbracket v(l) \rrbracket \leftarrow \llbracket u'(l) \rrbracket$  for each index  $l \in T_{i,j}$ 
  end for
end for
Return  $\llbracket v \rrbracket$ .
```

4.2 Sharing One-hot Vector

A one-hot vector is a vector with exactly one non-zero entry, whose value is 1. We define the “index” of a one-hot vector as the position where 1 locates. Sharing one-hot vectors is the first step toward constructing a shared permutation matrix, as a $k \times k$ permutation matrix consists of k pairwise distinct one-hot vectors.

Suppose party P_w intends to share a k -dimensional one-hot vector with index ind , where $k = 2^d$. First, P_w decomposes the index into $\log k = d$ bits $b_1, b_2, \dots, b_d$, such that

$$\text{ind} = \sum_{i=1}^d 2^{i-1} b_i.$$

Next, it secret shares $\llbracket b_i \rrbracket$ to all parties. To check that all shared values are binary, parties generate a random challenge $\lambda \in \mathbb{K}$ and verify that:

$$0 = \sum_{i=1}^d \lambda^{i-1} \llbracket b_i \rrbracket [1 - b_i].$$

This check evaluates a polynomial $F \in \mathbb{K}[X]$ of degree at most $d-1$ at a random point $\lambda \in \mathbb{K}$. Since a polynomial of degree $d-1$ has at most $d-1$ roots in $\mathbb{K}$, the probability that the check detects a non-binary b_i is overwhelming:

$$\Pr[F(\lambda) \neq 0 \mid \exists i \text{ s.t. } b_i(1 - b_i) \neq 0] \geq 1 - \frac{d-1}{|\mathbb{K}|} \geq 1 - \frac{d-1}{2^\kappa}.$$

This binary checking protocol is described in Algorithm 2.

If the binary check passes, parties convert the shared bits into a one-hot vector. This process of converting $\log k$ bits into k -long one-hot vector is known as “demultiplex”. In MPC, this can be done using the demux protocol proposed by Launchbury et al. [30], which computes the desired one-hot vector with $O(k)$ MPC multiplication in $O(\log \log k)$ rounds. The protocol works recursively: it splits the d bits into two subsets, generates a one-hot vector for each subset

Algorithm 2 $\text{BinCheck}(\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket)$

Ensure: Abort if any b_i is not in $\{0, 1\}$.

```

 $\lambda \leftarrow \Pi_{\text{challenge}}(\mathbb{K})$ 
 $\llbracket u \rrbracket \leftarrow \sum_{i=1}^d \lambda^{i-1} \llbracket b_i \rrbracket [1 - b_i]$  ▷ One call to  $\Pi_{\text{inner}}$ .
 $u \leftarrow \Pi_{\text{open}}(\llbracket u \rrbracket)$ 
All parties abort if  $u \neq 0$ .
```

via recursive calls, and then computes the tensor product of these two vectors to obtain the final one-hot vector with the correct index. For completeness, we present this demux protocol in Algorithm 3.

Algorithm 4 describes our OneHot protocol, which enables party P_w to share a one-hot vector. Notably, parties can verify all shared bits for multiple one-hot vectors with a single call to BinCheck by batching all their bit shares, rather than performing separate checks for each one-hot vector.

Algorithm 3 $(\llbracket s_1 \rrbracket, \dots, \llbracket s_{2^d} \rrbracket) \leftarrow \text{Demux}(\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket)$

Ensure: Output a one-hot vector with index $\sum_{i=1}^d 2^{i-1} b_i + 1$.

```

if  $d = 1$  then
  Return  $(1 - \llbracket b_1 \rrbracket, \llbracket b_1 \rrbracket)$ 
end if
 $t \leftarrow \lfloor d/2 \rfloor$ 
 $\llbracket u_1 \rrbracket, \llbracket u_2 \rrbracket, \dots, \llbracket u_{2^t} \rrbracket \leftarrow \text{Demux}(\llbracket b_1 \rrbracket, \dots, \llbracket b_t \rrbracket)$ 
 $\llbracket v_1 \rrbracket, \llbracket v_2 \rrbracket, \dots, \llbracket v_{2^{d-t}} \rrbracket \leftarrow \text{Demux}(\llbracket b_{t+1} \rrbracket, \dots, \llbracket b_d \rrbracket)$  ▷ Parallel recursive calls.
for  $i = 1$  to  $2^t$  do parallel
  for  $j = 1$  to  $2^{d-t}$  do parallel
     $\llbracket s_{i+2^t(j-1)} \rrbracket \leftarrow \Pi_{\text{mul}}(\llbracket u_i \rrbracket, \llbracket v_j \rrbracket)$ 
  end for
end for
Return  $\llbracket s_1 \rrbracket, \dots, \llbracket s_{2^d} \rrbracket$ 
```

Algorithm 4 $(\llbracket s_1 \rrbracket, \dots, \llbracket s_k \rrbracket) \leftarrow \text{OneHot}(k, P_w : \text{ind})$

Require: $k = 2^d$ is power-of-2.

Ensure: Output a one-hot vector, with index ind .

```

 $P_w$  computes bits  $b_1, \dots, b_d$ , s.t.  $\text{ind} = \sum_{i=1}^d 2^{i-1} b_i + 1$ .
 $P_w$  shares  $\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket$ .
BinCheck( $\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket$ )
Return Demux( $\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket$ ).
```

4.3 Sharing Permutation Matrix

A $k \times k$ permutation matrix consists of k one-hot row vectors with distinct indices. Thus, party P_w can share such a matrix by invoking the OneHot protocol k times and assembling the resulting row shares into a matrix P . While each row then contains exactly one 1, a corrupted P_w may render an ill-formed matrix by map multiple rows to the same column. So the parties must additionally verify that every column sums to 1.

Opening each column sum individually requires k openings. To batch the verification, the parties instead sample a random challenge

$\lambda \leftarrow \Pi_{\text{challenge}}(\mathbb{K})$, and check

$$\sum_{i=1}^k \left(\lambda^{i-1} \left(1 - \sum_{j=1}^k P(j, i) \right) \right) = 0.$$

Since a non-zero polynomial of degree $k - 1$ will have at most $k - 1$ roots in field, the equality will not hold with probability $1 - \frac{k-1}{|\mathbb{K}|} \geq 1 - \frac{k-1}{2^k}$ if any column has a sum different from 1.

Algorithms 5 and 6 detail the sharing and verification procedures. The entire process of Algorithm 6 consists of one call to $\Pi_{\text{challenge}}$ and one call to Π_{open} .

Algorithm 5 $\llbracket P \rrbracket \leftarrow \text{PermMat}(P_w : \pi)$

Require: π is a k -permutation, where k is power-of-2.

Ensure: Output a permutation matrix P .

for $i = 1$ to k **do parallel**

$v_i \leftarrow \text{OneHot}(k, P_w : \pi(i))$

end for

$\llbracket P \rrbracket := \llbracket v_{i,j} \rrbracket_{i,j \in [k]}$

$\text{PermCheck}(\llbracket P \rrbracket)$

Return $\llbracket P \rrbracket$.

Algorithm 6 $\text{PermCheck}(\llbracket P \rrbracket)$

Require: P is of size $k \times k$, with each row a one-hot vector.

Ensure: Abort with high probability if P is not a permutation matrix.

$\llbracket \text{sum} \rrbracket \leftarrow \llbracket 0 \rrbracket$

$\lambda \leftarrow \Pi_{\text{challenge}}(\mathbb{K})$

for $i = 1$ to k **do** $\triangleright$ Enumerate columns.

$\llbracket \text{sum} \rrbracket \leftarrow \llbracket \text{sum} \rrbracket + \lambda^{i-1} (1 - \sum_{j=1}^k \llbracket P(j, i) \rrbracket)$

end for

$\text{sum} \leftarrow \Pi_{\text{open}}(\llbracket \text{sum} \rrbracket)$

Abort if $\text{sum} \neq 0$.

4.4 Decomposing and Sharing Permutation

The above protocols allow the parties to share a $k \times k$ permutation matrix. To share an m -permutation for large m , we first represent it using many k -permutations. This is achieved using the Beneš network.

The Beneš network, due to the seminal work of Beneš [9], is a switching network capable of representing any m -permutation with $O(\log m)$ depth. By partitioning the network into layers and extracting intermediate permutations, one obtains a natural decomposition of the original permutation. Chase et al. [12] introduced a permutation decomposition technique based on this reformulation, decomposing an m -permutation into $O(\frac{m \log m}{k \log k})$ smaller permutations, each of size k . The classic Beneš network requires m to be a power of 2. Although there exist generalizations to arbitrary m , they lose some important properties, such as the independence of the small decomposed permutations, which is crucial for the round complexity remaining $O(\log m)$.

To formalize this process, let

$$(\pi_1, \pi_2, \dots, \pi_s) \leftarrow \text{Decompose}(\pi, k)$$

denote the process of decomposing π such that

$$\pi = \pi_s \circ \pi_{s-1} \circ \dots \circ \pi_1,$$

where each π_i is an m -permutation. According to Chase et al. [12], the concrete value is $s = (2 \log m - 1) / \log k = O(\frac{\log m}{\log k})$. Each π_i can be further decomposed as $\{\pi_{i,j}\}_{j \in [m/k]}$, where each $\pi_{i,j}$ is an m -permutation that affects only k entries. Thus,

$$\pi = \pi_{s,1} \circ \pi_{s,2} \circ \dots \circ \pi_{s,m/k} \circ \pi_{s-1,1} \circ \dots \circ \pi_{1,m/k}.$$

Define the “task” of each $\pi_{i,j}$ as

$$\text{Task}(m, k, i, j) = \{x \in [m] \mid \pi_{i,j} \text{ affects the } x\text{-th entry}\}.$$

Let $\pi'_{i,j}$ be the compressed (i.e., k -permutation) version of $\pi_{i,j}$ acting only on its task. The decomposition guarantees that each task has size k .

A key property of this decomposition is its task invariance: the task of $\pi_{i,j}$ is independent of the original permutation π . This allows all parties to pre-agree on the tasks (i.e., which entries to permute) without learning anything about the specific permutation.

Example 4.2. Consider decomposing an 8-permutation π with $k = 4$. Suppose π is decomposed and we obtain

$$\pi_1 = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 5 & 4 & 3 & 6 & 7 & 8 & 1 & 2 \end{pmatrix}.$$

It can be further decomposed into two disjoint permutations, i.e.

$$\pi_{1,1} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 5 & 2 & 3 & 4 & 7 & 6 & 1 & 8 \end{pmatrix},$$

$$\pi_{1,2} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 4 & 3 & 6 & 5 & 8 & 7 & 2 \end{pmatrix}.$$

$\pi_{1,1}$ and $\pi_{1,2}$ are disjoint 8-permutations, in the sense that each acts on disjoint fixed subsets (i.e., their tasks) of $\{1, \dots, 8\}$:

$$\text{Task}(8, 4, 1, 1) = \{1, 3, 5, 7\},$$

$$\text{Task}(8, 4, 1, 2) = \{2, 4, 6, 8\}.$$

Although $\pi_{1,1}$ may vary with π , its task does not, due to the task-invariant property. We may compress them into $\pi'_{1,1}$ and $\pi'_{1,2}$:

$$\pi'_{1,1} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 2 & 4 & 1 \end{pmatrix}, \quad \pi'_{1,2} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 1 \end{pmatrix}.$$

Thus, π_1 is represented by two disjoint 4-permutations $\pi'_{1,1}$ and $\pi'_{1,2}$.

Our permutation-sharing protocol is given in Algorithm 7. The parties first agree on the parameter k and locally compute a sequence of tasks $\{\text{Task}(m, k, i, j)\}$. The dealer P_w chooses a permutation π and locally decomposes π into the small permutations $\{\pi'_{i,j}\}$. Then P_w shares each as a permutation matrix. Since the permutations $\{\pi_{i,j}\}$ for a fixed i are disjoint, this yields a valid k -shared m -permutation.

Algorithm 7 $\llbracket \pi \rrbracket_k \leftarrow \text{SharePerm}(k, P_w : \pi)$ **Require:** π is an m -permutation.**Ensure:** Return a k -shared m -permutation $\llbracket \pi \rrbracket_k$.Party P_w let $(\pi_1, \dots, \pi_s) \leftarrow \text{Decompose}(\pi, k)$.Party P_w further decomposes each π_i into $\pi_{i,1}, \dots, \pi_{i,m/k}$.**for** $i = 1$ to s **do parallel****for** $j = 1$ to $\lceil m/k \rceil$ **do parallel** $T_{i,j} \leftarrow \text{Task}(m, k, i, j)$ $\llbracket P_{\pi'_{i,j}} \rrbracket \leftarrow \text{PermMat}(P_w : \pi'_{i,j})$ **end for****end for**Return $\llbracket \pi \rrbracket_k := (k, s, \{\{\llbracket P_{\pi'_{i,j}} \rrbracket, T_{i,j}\}_{j \in \lceil m/k \rceil}\}_{i \in [s]})$.

4.5 Shuffle Protocol

Our first shuffle protocol consists of exactly n calls to the permutation protocol developed above, following a permute-in-turn mechanism where parties apply their permutations sequentially. The two phases of the protocol are formally presented in Algorithm 8 and Algorithm 9. In the offline phase, parties prepare n shared permutations, one from each party. In the online phase, parties apply these permutations to the incoming data by protocol `Permute`.

Algorithm 8 $\{\llbracket \pi_i \rrbracket_k\}_{i \in [n]} \leftarrow \text{Shuffle1Off}(k, m)$ **Require:** Both m and k are power-of-2.**for** $w = 1$ to n **do**Party P_w draws m -permutation π_w uniformly. $\text{SharePerm}(k, P_w : \pi_w)$ **end for**Return $\{\llbracket \pi_i \rrbracket_k\}_{i \in [n]}$.**Algorithm 9** $\llbracket v'_1, v'_2, \dots, v'_l \rrbracket \leftarrow \text{Shuffle1On}(\llbracket v_1, v_2, \dots, v_l \rrbracket)$ **Require:** Each v_i is of length m , which is power-of-2.**Ensure:** Return shuffled $\llbracket \pi(v_i) \rrbracket_{i \in [l]}$, with a secret uniform π .Fetch fresh $\{\llbracket \pi_i \rrbracket\}_{i \in [n]}$ generated by `Shuffle1Off`.**for** $w = 1$ to n **do****for** $i = 1$ to l **do parallel** $\llbracket v_i \rrbracket \leftarrow \text{Permute}(\llbracket \pi_w \rrbracket, \llbracket v_i \rrbracket)$ **end for****end for**Return $\llbracket v'_1, v'_2, \dots, v'_l \rrbracket$.

4.6 Complexity Analysis

The following theorems analyze the complexity of the key protocols introduced in this section. Their proofs, together with the complexity analyses of the remaining protocols referenced above, are provided in Appendix B.1. The complexity of the shuffle protocols, with a clear separation between offline and online costs, is summarized in Theorem 4.4.

THEOREM 4.3. *For protocol $\text{SharePerm}(k, P_w : \pi)$ given in Algorithm 7, the communication, computation and round complexity is $O(\frac{k}{\log k} nm \log m)$, $O(\frac{k}{\log k} nm \log m)$ and $O(\log \log k)$, respectively.*

THEOREM 4.4. *Consider an input collection $\llbracket \mathbf{V} \rrbracket = \llbracket \mathbf{v}_1, \dots, \mathbf{v}_m \rrbracket$ of m vectors, each of length l , to be jointly shuffled by the parties. For our first shuffle protocol described in Algorithm 8 and 9, the offline communication, computation and round complexity is $O(\frac{k}{\log k} n^2 m \log m)$, $O(\frac{k}{\log k} n^2 m \log m)$ and $O(\log \log k)$, respectively. The online communication, computation and round complexity is $O(\frac{l}{\log k} n^2 m \log m)$, $O(\frac{kl}{\log k} n^2 m \log m)$ and $O(\frac{n \log m}{\log k})$, respectively.*

5 Shuffle Protocol with Linear Online Communication

In this section, we present our second shuffle protocol. By leveraging the maliciously secure shuffle correlation technique of Gao et al. [18], we obtain a shuffle protocol with $O(nml)$ online communication and $O(\frac{k+l}{\log k} n^2 m \log m)$ offline communication. For clarity of exposition, we first describe the construction for the case $l = 1$, that is, shuffling individual vector entries.

At a high level, the semi-honest shuffle correlation operates as follows. Suppose the parties share a vector $\llbracket \mathbf{x} \rrbracket$ and an independently random vector $\llbracket \mathbf{r} \rrbracket$. Each party P_i samples a random permutation π_i and secret-shares a random vector $\llbracket \mathbf{r}_i \rrbracket$ among all parties. In the online phase, the parties first open $\llbracket \mathbf{y}_0 \rrbracket = \llbracket \mathbf{x} \rrbracket - \llbracket \mathbf{r} \rrbracket$ to P_1 . Subsequently, each party P_i sends to P_{i+1} the value

$$\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1}) + \mathbf{r}_i,$$

while the last party P_n broadcasts $\mathbf{y}_n$, where

$$\mathbf{y}_n = \pi_n(\pi_{n-1}(\dots \pi_1(\mathbf{x} + \mathbf{r}) + \mathbf{r}_1 \dots) + \mathbf{r}_{n-1}) + \mathbf{r}_n.$$

To recover the shuffled output, the parties jointly remove the random mask by subtracting

$$\llbracket \Delta \rrbracket = \pi_n(\pi_{n-1}(\dots \pi_1(\llbracket \mathbf{r} \rrbracket) + \llbracket \mathbf{r}_1 \rrbracket \dots) + \llbracket \mathbf{r}_{n-1} \rrbracket) + \mathbf{r}_n$$

from $\mathbf{y}_n$.

Accordingly, the primary objective of the offline phase is to generate the correction term $\llbracket \Delta \rrbracket$. Gao et al. [18] show that this can be accomplished efficiently using an ideal permutation functionality

$$\llbracket \pi(\mathbf{x}) \rrbracket \leftarrow \Pi_{\text{perm}}(P_i : \pi, \llbracket \mathbf{x} \rrbracket),$$

which can be instantiated by our permutation protocol in Section 4. They further demonstrate how to enhance the construction to achieve malicious security (i.e., security with abort) by jointly shuffling $\beta \mathbf{x}$ alongside $\mathbf{x}$ and checking the correctness of messages.

For completeness, we provide a full specification of the protocol below. As noted above, the resulting protocols initially achieve security with abort. In Section 6, we show how to further enhance them to achieve guaranteed output delivery (GOD).

5.1 Correlation Check of Public Values

The goal of the protocol `CorrCheck(a, b,  $\llbracket \beta \rrbracket, \llbracket \mathbf{r} \rrbracket$ )` is to allow a designated party P_w to check whether the relation $\beta \mathbf{a} = \mathbf{b} + \mathbf{r}$ holds. For an honest P_w , the protocol aborts with overwhelming probability whenever the correlation is violated.

This protocol is used in the online phase of the shuffle protocol, where each party P_i sends vectors $\mathbf{a}$ and $\mathbf{b}$ to P_{i+1} . Party P_{i+1} can invoke `CorrCheck` to ensure that the received messages are correct. If an honest P_{i+1} accepts, this guarantees that P_i has sent correct

messages. As shown in [18], as long as each honest P_w receives correct messages, the permutation chosen by P_w remains hidden.

To perform the check, party P_w first samples a random challenge $\lambda \leftarrow \mathbb{K}$ locally. It then computes

$$u \leftarrow \sum_{i=1}^m \lambda^{i-1} \mathbf{a}(i), \quad v \leftarrow \sum_{i=1}^m \lambda^{i-1} \mathbf{b}(i),$$

and broadcasts λ, u, v . All parties then jointly compute

$$[s] \leftarrow [\beta] \cdot u - v - \sum_{i=1}^m \lambda^{i-1} [\mathbf{r}(i)],$$

and open $[s]$ to check whether it is zero. Apart from the broadcast of λ , the only communication comes from calls to Π_{mul} and Π_{open} .

The protocol is formally described in Algorithm 10.

Algorithm 10 CorrCheck($P_w, \mathbf{a}, \mathbf{b}, [\beta], [\mathbf{r}]$)

Require: $\mathbf{a}$ and $\mathbf{b}$ are not secret and known to P_w .

Ensure: If $P_w = \emptyset$ or P_w is honest, abort w.h.p. if $\beta \mathbf{a} \neq \mathbf{b} + \mathbf{r}$.

Let m be the length of $\mathbf{a}, \mathbf{b}$ and $[\mathbf{r}]$.

if $P_w \neq \emptyset$ **then**

P_w draws uniformly $\lambda \leftarrow \mathbb{K}$.

P_w computes $u \leftarrow \sum_{i=1}^m \lambda^{i-1} \mathbf{a}(i)$.

P_w computes $v \leftarrow \sum_{i=1}^m \lambda^{i-1} \mathbf{b}(i)$.

P_w broadcasts λ, u, v .

else ▷ $\mathbf{a}$ and $\mathbf{b}$ are known to all.

$\lambda \leftarrow \Pi_{\text{challenge}}()$

All parties locally compute $u \leftarrow \sum_{i=1}^m \lambda^{i-1} \mathbf{a}(i)$.

All parties locally compute $v \leftarrow \sum_{i=1}^m \lambda^{i-1} \mathbf{b}(i)$.

end if

$[s] \leftarrow [\beta]u - v - \sum_{i=1}^m \lambda^{i-1} [\mathbf{r}(i)]$

$[c] \leftarrow \Pi_{\text{rand}}(\mathbb{K})$

$[s] \leftarrow \Pi_{\text{mul}}([s], [c])$

$s \leftarrow \Pi_{\text{open}}([s])$

All parties abort if $s \neq 0$.

5.2 Offline Phase

The main task of the offline phase is to generate a shuffle correlation, i.e., a collection of random values satisfying specific correlation constraints. The definition of shuffle correlation is given below.

Definition 5.1 (m -Shuffle Correlation[18]). The m -shuffle correlation is defined as

$$\text{cor} = \left\{ \begin{array}{cccc} [\beta] & [\mathbf{r}] & [\beta \mathbf{r}] & [s] \\ \pi_1 & \pi_2 & \cdots & \pi_n \\ [\pi_1(\mathbf{r}'_1)] & [\pi_2(\mathbf{r}'_2)] & \cdots & [\pi_n(\mathbf{r}'_n)] \\ \mathbf{z}_2 & \cdots & \mathbf{z}_n & \end{array} \right\},$$

where

- (1) Each π_i is an m -permutation chosen by party P_i and known only to it; if P_i is honest, then π_i is uniform over all m -permutations.
- (2) $[\beta]$ is a uniformly random secret-shared element of $\mathbb{K}$.
- (3) $[\mathbf{r}], [s], [\mathbf{r}'_1], \dots, [\mathbf{r}'_n]$ are secret-shared vectors of length m , with each entry independently uniformly random over $\mathbb{K}$. $[\beta \mathbf{r}]$ is secret-shared $\beta \cdot \mathbf{r}$.

- (4) $\mathbf{z}_i = (\mathbf{z}_{i,1}, \mathbf{z}_{i,2})$, where $\mathbf{z}_{i,j}$ is a vector of length m . The entries of $\mathbf{z}_{i,1}$ are uniformly random, under the constraint

$$\begin{cases} \mathbf{z}_{i,2} = \beta \mathbf{z}_{i,1} + \pi_{i-1}(\mathbf{r}'_{i-1}) - \mathbf{r}'_i & \forall i = 2, 3, \dots, n \\ \mathbf{s} = \pi_n(\pi_{n-1}(\cdots \pi_2(\pi_1(\mathbf{r}) - \mathbf{z}_{2,1}) - \mathbf{z}_{3,1} \cdots) - \mathbf{z}_{n,1}) \end{cases}$$

To generate a shuffle correlation, the parties first generate a random $[\beta]$ and $2n$ random vectors of length m over $\mathbb{K}$, denoted as

$$[\mathbf{r}_1], [\mathbf{r}_2], \dots, [\mathbf{r}_n], [\mathbf{r}'_1], [\mathbf{r}'_2], \dots, [\mathbf{r}'_n].$$

The parties then call Π_{mul} to compute the entry-wise scalar product of $[\beta]$ with each $[\mathbf{r}_i]$, yielding

$$[\beta \mathbf{r}_1], [\beta \mathbf{r}_2], \dots, [\beta \mathbf{r}_n],$$

The parties then invoke Permute n times (for $i \in [n]$) to compute

$$[\pi_i(\mathbf{r}_i), \pi_i(\beta \mathbf{r}_i), \pi_i(\mathbf{r}'_i)] \leftarrow \text{Permute}(P_i : \pi_i, [\mathbf{r}_i, \beta \mathbf{r}_i, \mathbf{r}'_i]).$$

For each $i \geq 2$, the parties compute

$$[\mathbf{z}_{i,1}] \leftarrow [\pi_{i-1}(\mathbf{r}_{i-1})] - [\mathbf{r}_i],$$

$$[\mathbf{z}_{i,2}] \leftarrow [\pi_{i-1}(\beta \mathbf{r}_{i-1})] + [\pi_{i-1}(\mathbf{r}'_{i-1})] - [\beta \mathbf{r}_i] - [\mathbf{r}'_i].$$

Note that $[\mathbf{z}_i] := ([\mathbf{z}_{i,1}], [\mathbf{z}_{i,2}])$ is a vector of length $2m$.

The parties then open each $[\mathbf{z}_i]$ to P_i , for each $i \geq 2$. The shuffle correlation returned by this protocol is

$$\text{cor} := \left\{ \begin{array}{cccc} [\beta] & [\mathbf{r}_1] & [\beta \mathbf{r}_1] & [\pi_n(\mathbf{r}_n)] \\ \pi_1 & \pi_2 & \cdots & \pi_n \\ [\pi_1(\mathbf{r}'_1)] & [\pi_2(\mathbf{r}'_2)] & \cdots & [\pi_n(\mathbf{r}'_n)] \\ \mathbf{z}_2 & \cdots & \mathbf{z}_n & \end{array} \right\}.$$

Note that each $\mathbf{z}_i$ is held as plaintext by party P_i for $i \geq 2$, and variables with bracket is shared among all parties.

This protocol is formally described in Algorithm 11.

Algorithm 11 cor $\leftarrow$ Shuffle2_{off}($P_1 : \pi_1, \dots, P_n : \pi_n$)

Require: For honest P_i , π_i is sampled uniformly from all m -permutations.

Ensure: Output a shuffle correlation.

$[\beta] \leftarrow \Pi_{\text{rand}}(\mathbb{K})$.

for $i = 1$ to n **do parallel**

$[\mathbf{r}_i(j)] \leftarrow \Pi_{\text{rand}}(\mathbb{K})$ for $j \in [m]$

$[\mathbf{r}'_i(j)] \leftarrow \Pi_{\text{rand}}(\mathbb{K})$ for $j \in [m]$

$[\beta \mathbf{r}_i] \leftarrow \Pi_{\text{mul}}([\beta], [\mathbf{r}_i])$

$[\pi_i] \leftarrow \text{SharePerm}(k, P_i : \pi_i)$

$[\pi_i(\mathbf{r}_i), \pi_i(\beta \mathbf{r}_i), \pi_i(\mathbf{r}'_i)] \leftarrow \text{Permute}([\pi_i], [\mathbf{r}_i, \beta \mathbf{r}_i, \mathbf{r}'_i])$

if $i \geq 2$ **then**

$[\mathbf{z}_{i,1}] \leftarrow [\pi_{i-1}(\mathbf{r}_{i-1})] - [\mathbf{r}_i]$

$[\mathbf{z}_{i,2}] \leftarrow [\pi_{i-1}(\beta \mathbf{r}_{i-1})] - [\beta \mathbf{r}_i] + [\pi_{i-1}(\mathbf{r}_{i-1})] - [\mathbf{r}'_i]$

$[\mathbf{z}_i] := ([\mathbf{z}_{i,1}], [\mathbf{z}_{i,2}])$

end if

end for

for $i = 2$ to n **do parallel**

$\mathbf{z}_i \leftarrow \Pi_{\text{open}}([\mathbf{z}_i], P_i)$

end for

$$\text{Return cor} := \left\{ \begin{array}{cccc} [\beta] & [\mathbf{r}_1] & [\beta \mathbf{r}_1] & [\pi_n(\mathbf{r}_n)] \\ \pi_1 & \pi_2 & \cdots & \pi_n \\ [\pi_1(\mathbf{r}'_1)] & [\pi_2(\mathbf{r}'_2)] & \cdots & [\pi_n(\mathbf{r}'_n)] \\ \mathbf{z}_2 & \cdots & \mathbf{z}_n & \end{array} \right\}.$$

5.3 Online Phase

In the online phase, to shuffle $\llbracket \mathbf{x} \rrbracket$, the parties first compute

$$\llbracket \mathbf{z}_1 \rrbracket \leftarrow (\llbracket \mathbf{x} \rrbracket - \llbracket \mathbf{r}_1 \rrbracket, \llbracket \beta \rrbracket \llbracket \mathbf{x} \rrbracket - \llbracket \beta \mathbf{r}_1 \rrbracket - \llbracket \mathbf{r}'_1 \rrbracket),$$

and open $\llbracket \mathbf{z}_1 \rrbracket$ to P_1 . P_1 applies its permutation and sends

$$\mathbf{y}_1 \leftarrow \pi_1(\mathbf{z}_1)$$

to party P_2 . Here, we slightly abuse the notation: we define $\pi_1(\mathbf{z}_1) := (\pi_1(\mathbf{z}_{1,1}), \pi_1(\mathbf{z}_{1,2}))$. This notation extends to all such vectors of length $2m$, while π is an m -permutation.

For $2 \leq i \leq n-1$, P_i first checks if the message received from P_{i-1} is correct, by invoking

$$\text{CorrCheck}(P_i, \mathbf{y}_{i-1,1}, \mathbf{y}_{i-1,2}, \llbracket \beta \rrbracket, \llbracket \pi_{i-1}(\mathbf{r}'_{i-1}) \rrbracket).$$

If the check passes, P_i computes

$$\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1} + \mathbf{z}_i)$$

and sends $\mathbf{y}_i$ to P_{i+1} ; otherwise, all parties abort.

The last party P_n performs the same check on $\mathbf{y}_{n-1}$. If the check passes, P_n broadcasts $\mathbf{y}_n \leftarrow \pi_n(\mathbf{y}_{n-1} + \mathbf{z}_n)$. Then all parties invoke

$$\text{CorrCheck}(\emptyset, \mathbf{y}_{n,1}, \mathbf{y}_{n,2}, \llbracket \beta \rrbracket, \llbracket \pi_n(\mathbf{r}'_n) \rrbracket).$$

If the final check passes, the protocol outputs the secret-shared shuffled vector by computing

$$\llbracket \mathbf{x}' \rrbracket \leftarrow \mathbf{y}_{n,1} + \llbracket \pi_n(\mathbf{r}_n) \rrbracket.$$

The above process is formally described in Algorithm 12. To see the correctness of the protocol, note that if all parties are honest, each P_{i+1} will receive $\mathbf{y}_i$ satisfying

$$\begin{aligned} \mathbf{z}_{i+1,1} &= \pi_i(\mathbf{r}_i) - \mathbf{r}_{i+1}, \\ \mathbf{z}_{i+1,2} &= \beta \mathbf{z}_{i+1,1} + \pi_i(\mathbf{r}'_i) - \mathbf{r}'_{i+1}, \\ \mathbf{y}_{i,1} &= \pi_i \circ \dots \circ \pi_1(\mathbf{x}) - \pi_i(\mathbf{r}_i), \\ \mathbf{y}_{i,2} &= \beta \mathbf{y}_{i,1} - \pi_i(\mathbf{r}'_i). \end{aligned}$$

It will send $\mathbf{y}_{i+1} = \pi_{i+1}(\mathbf{y}_i + \mathbf{z}_{i+1})$. $\mathbf{y}_i$ maintains the invariant

$$\begin{aligned} \mathbf{y}_{i+1,1} &= \pi_{i+1} \circ \dots \circ \pi_1(\mathbf{x}) - \pi_{i+1}(\mathbf{r}_{i+1}), \\ \mathbf{y}_{i+1,2} &= \beta \mathbf{y}_{i+1,1} - \pi_{i+1}(\mathbf{r}'_{i+1}). \end{aligned}$$

Fianlly, parties compute $\mathbf{y}_{n,1} + \pi_n(\mathbf{r}_n)$, which is exactly $\pi(\mathbf{x})$.

Security relies on the secrecy of β and $\mathbf{r}'$: a corrupted party cannot forge an invalid message and passes the correlation check except with non-negligible probability. Hence, whenever a check passes, the corresponding input must be correct. This is formally proven in [18].

5.4 Complexity Analysis

We analyze the complexity of the above shuffle protocol, distinguishing explicitly between its offline and online phases. The main results are stated below, and the proofs are provided in Appendix B.2.

THEOREM 5.2. *For $\text{Shuffle2}_{\text{off}}(P_1 : \pi_1, \dots, P_n : \pi_n)$ described in Algorithm 11, the (offline) communication, computation and round complexity is $O(\frac{k}{\log k} n^2 m \log m)$, $O(\frac{k}{\log k} n^2 m \log m)$ and $O(\frac{\log m}{\log k} + \log \log k)$, respectively.*

Algorithm 12 $\llbracket \mathbf{x}' \rrbracket \leftarrow \text{Shuffle2}_{\text{on}}(\llbracket \mathbf{x} \rrbracket)$

Ensure: Output shuffled $\llbracket \pi(\mathbf{x}) \rrbracket$ for some secret uniform π .

Fetch a fresh m -shuffle correlation cor generated by $\text{Shuffle2}_{\text{off}}$.

$\llbracket \mathbf{z}_1 \rrbracket \leftarrow (\llbracket \mathbf{x} \rrbracket - \llbracket \mathbf{r}_1 \rrbracket, \llbracket \beta \rrbracket \llbracket \mathbf{x} \rrbracket - \llbracket \beta \mathbf{r}_1 \rrbracket - \llbracket \mathbf{r}'_1 \rrbracket)$

Open $\llbracket \mathbf{z}_1 \rrbracket$ to P_1 .

P_1 computes $\mathbf{y}_1 \leftarrow \pi_1(\mathbf{z}_1)$.

P_1 sends $\mathbf{y}_1$ to P_2 .

for $i = 2$ to n **do**

$\text{CorrCheck}(P_i, \mathbf{y}_{i-1,1}, \mathbf{y}_{i-1,2}, \llbracket \beta \rrbracket, \llbracket \pi_{i-1}(\mathbf{r}'_{i-1}) \rrbracket)$

P_i computes $\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1} + \mathbf{z}_i)$.

if $i < n$ **then**

P_i sends $\mathbf{y}_i$ to P_{i+1} .

else

P_n broadcasts $\mathbf{y}_n$.

end if

end for

$\text{CorrCheck}(\emptyset, \mathbf{y}_{n,1}, \mathbf{y}_{n,2}, \llbracket \beta \rrbracket, \llbracket \pi_n(\mathbf{r}'_n) \rrbracket)$

Return $\mathbf{y}_{n,1} + \llbracket \pi_n(\mathbf{r}_n) \rrbracket$.

THEOREM 5.3. *For $\text{Shuffle2}_{\text{on}}(\llbracket \mathbf{v} \rrbracket)$ described in Algorithm 12, the communication, computation and round complexity is $O(nm + n^2)$, $O(n^2 m)$ and $O(n)$, respectively. Note that for $n = O(m)$, the communication is just $O(nm)$.*

COROLLARY 5.4. *For shuffling rows of $m \times l$ matrix, the shuffle protocol described in Algorithm 11 and 12 requires $O(nml + n^2)$ online communication, $O(n^2 ml)$ online computation and $O(n)$ online rounds, with an offline overhead of $O(\frac{k+l}{\log k} n^2 m \log m)$ communication, $O(\frac{kl}{\log k} n^2 m \log m)$ computation and $O(\frac{\log m}{\log k} + \log \log k)$ rounds.*

This concludes our main results in this section.

6 Guaranteed Output Delivery

This section focuses on enhancing the secure-with-abort protocols beforementioned to achieve guaranteed output delivery (GOD), a critical security property that ensures honest parties obtain correct outputs regardless of corrupted parties' behavior. Our high-level approach is as follows. For protocols constructed purely from standard MPC primitives, GOD can be achieved directly by replacing secure-with-abort components with their GOD-enabled counterparts (e.g., [19]). For the more specialized scenario — namely, the online phase of the second shuffle protocol, where parties perform local computations and exchange intermediate results — we further leverage the dispute control technique proposed by Beerliova-Trubiniova and Hirt [6] to achieve GOD. Consequently, all protocols proposed in this work are enhanced from secure-with-abort to GOD protocols.

We next present the two key ingredients of our construction. First, our scheme fully supports corruption identification, which in turn enables achieving GOD. Second, our scheme leverages the authentication tag technique of Ben-Sasson et al. [8], thereby enabling the use of circuit division to achieve GOD with an overhead linear in the circuit size. In particular, authentication tags act as commitments to the shares produced within each circuit segment, allowing honest parties to recover a correct intermediate state in the presence of adversarial deviations, rather than restarting. Consequently, our GOD-enabled SLIDE achieves linear online communication and

rounds, matching that of the shuffle with linear online communication we proposed in Chapter 5. Additionally, we introduce further optimizations for our GOD-enabled protocols.

6.1 Corruption Identification

The key to achieving GOD lies identifying corrupted parties and subsequently exclude them from further computation. We now show that our construction is robust enough to support corruption identification, in the sense that any failure of a correctness check will expose either a new corrupted party or a new disputing pair.

The first step is to equip all underlying primitives with corruption identification. Concretely, each primitive supported by the ideal functionality F_{Shamir} is enhanced so that, upon failure, it reveals new corrupted parties or disputing pairs, rather than simply aborting. For example, the multiplication protocol $\Pi_{\text{mul}}(\llbracket a \rrbracket, \llbracket b \rrbracket)$ now terminates in one of the following three outcomes:

- (1) The multiplication succeeds, and all honest parties hold consistent shares of $\llbracket ab \rrbracket$.
- (2) The multiplication fails and publicly identifies a new corrupted party $P_i \notin \text{Corr}$.
- (3) The multiplication fails and publicly identifies a new disputing pair $\{P_i, P_j\} \notin \text{Disp}$.

It is crucial that any reported corrupted party or disputing pair is new, so that the size of either Corr or Disp strictly increases. Moreover, these primitives are sound: an honest party is never falsely identified as corrupted, nor will two honest parties ever be put into dispute. For primitives with public input, deviating from the publicly agreed input is also detected as a failure. Such primitives are provided by Goyal et al. [19].

We now argue that all protocols developed so far can be extended to support corruption identification, by replacing “abort” with identification of corrupted parties or disputing pairs.

Recall that the generation of one-hot vectors relies on BinCheck (Algorithm 2), Demux (Algorithm 3), and OneHot (Algorithm 4). These protocols only invoke basic MPC primitives and recursively call each other; therefore, they directly inherit corruption identification.

For permutation sharing, we developed PermMat (Algorithm 5), PermCheck (Algorithm 6), SharePerm (Algorithm 7), and Permute (Algorithm 1). Protocols PermMat and Permute consist solely of calls to previously established primitives. If PermCheck fails because the final check value is nonzero (as opposed to a failure in a subroutine), then the party P_w who specified the permutation matrix must be corrupted. Similarly, although the decomposition in SharePerm is performed locally by P_w , any malformed permutation will be detected during PermMat and PermCheck, thereby identifying P_w as corrupted. Conversely, if all checks pass, then the reconstructed permutation π is well-formed and serves as a valid input for P_w .

Building on these enhanced protocols, the first shuffle protocol introduced in Section 4 therefore supports corruption identification.

For the shuffle protocol with linear communication complexity, we further developed CorrCheck (Algorithm 10), Shuffle2_{off} (Algorithm 11), and Shuffle2_{on} (Algorithm 12). If CorrCheck fails due to a primitive failure, corrupted parties are identified immediately.

The offline protocol again consists only of calls to primitives that already support corruption identification.

When CorrCheck fails as a nonzero value $s \neq 0$ is revealed, the misbehaving party is not immediately determined. However, during Shuffle2_{on}, if party P_i observes such a failure, it immediately enters a dispute with P_{i-1} . Indeed, as shown in Gao et al. [18], if any party P_j misbehaves by sending an incorrect message y_j to an honest P_{j+1} , the correlation check at P_{j+1} fails with overwhelming probability. Since all previous checks have passed, P_{i-1} must have provided correct inputs to CorrCheck. Thus, a current failure implies either P_{i-1} sent an incorrect y_{i-1} to P_i , or P_i used an incorrect y_{i-1} in the correlation check; hence, at least one of P_{i-1} or P_i is corrupted.

If P_{i-1} and P_i are in dispute, the value y_{i-1} must be relayed through a third party $P_{(i-1) \leftrightarrow i}$ that does not dispute with either. Letting $P_{(i-1) \leftrightarrow i}$ access y_{i-1} does not compromise security, since the analysis in Gao et al. [18] shows that security is preserved even if all y_i values are broadcast (broadcasting is avoided only to reduce communication overhead). If the correlation check for P_i fails again, the relay must publicly dispute with either P_{i-1} or P_i , thereby increasing Disp.

Finally, if the last correlation check fails, then P_n must be corrupted, as all prior checks have succeeded and P_n must have received correct inputs.

In summary, all protocols proposed in this paper support corruption identification. Consequently, they can be strengthened to achieve GOD.

6.2 Authentication Tag

It remains to show that the technique of authentication tag [8] presented below is applicable to our constructions to fully achieve GOD with the worst computation linear in circuit size.

Authentication Tag. To enable party P_w to authenticate its share of a secret $\llbracket x \rrbracket$, the parties first decompose the secret as

$$\llbracket x \rrbracket = \sum_{i=1}^n \llbracket x(i) \rrbracket,$$

where $\llbracket x(i) \rrbracket$ is shared earlier by party P_i . Such a decomposition is always possible since all secrets in our protocols are obtained via linear combinations of previously shared values.

For each $\llbracket x(i) \rrbracket$, party P_w obtains one authentication tags from all parties (denoted as verifier P_v) not disputing with it. Specifically, each verifier P_v samples random $\mu_{v \rightarrow w}, u_{v \rightarrow w} \in \mathbb{K}$ and assists P_w in computing

$$\tau_{v \rightarrow w} := \mu_{v \rightarrow w} \odot \mathbf{x}_w(i) + u_{v \rightarrow w},$$

where $\mathbf{x}_w(i)$ denotes P_w 's share of $\mathbf{x}(i)$ and $\odot$ denotes the inner product. This computation is carried out using a mini-MPC protocol [8], which is compatible with Shamir secret sharing.

Without knowledge of $(\mu_{v \rightarrow w}, u_{v \rightarrow w})$, P_w can forge a valid authentication tag only with probability $2^{-\kappa}$. Consequently, if an inconsistency in $\llbracket x \rrbracket$ is later detected and traced to $\llbracket x(i) \rrbracket$, party P_i must accuse some party P_j of misbehavior. Upon accusation, P_j reveals $\mathbf{x}_j(i)$ together with the corresponding authentication tags to all verifiers. Each verifier independently checks the tag and votes for either P_i or P_j as corrupted; a majority vote identifies a corrupted party. Revealing $\mathbf{x}_j(i)$ is safe, since at least one of the two parties is corrupted.

This technique is applicable to all our protocols, as they satisfy the following property: upon termination, all honest parties hold consistent shares. Indeed, all shared values originate either from Π_{share} or from linear combinations thereof. Any misbehavior occurring after protocol completion is therefore handled by the authentication tag mechanism, which suffices to achieve GOD with linear overhead.

6.3 Optimizations

Let $P_{\text{active}} = [n] \setminus \text{Corr}$ denote the set of parties remaining in the computation. As observed in [19], communication can be further reduced by randomly selecting a subset $T \subseteq P_{\text{active}}$ of size $t + 1$ and letting only parties in T perform most of the computation. This is sound since $t + 1$ shares suffice to reconstruct a secret, and at least one of these parties is honest. The same optimization applies to our shuffle protocols. Instead of having all parties permute in turn, under an honest-majority assumption, it suffices to let only $t + 1$ parties apply permutations sequentially to ensure the uniformity of the resulting shuffle. Consequently, the shuffle correlation can be generated for a smaller set of parties, leading to reduced communication and computation costs.

We further remark that the offline and online phases of a shuffle protocol can be cleanly separated into multiple segments. This is because all secret-shared values in the correlation remain information-theoretically hidden even if CorrCheck fails: the opened value $\llbracket s \rrbracket$ is either zero or a uniformly random element, and thus reveals no information. As a result, a shuffle correlation can be safely reused after a failure.

Finally, even seemingly atomic protocols at a high level (e.g., $\text{Shuffle}_{2\text{off}}$) admit decomposition into smaller computation segments, since they ultimately consist of calls to basic MPC primitives and thus induce a divisible circuit structure, allowing partitioning without compromising correctness or integrity, enabling further flexibility in implementation.

7 Conclusion

In this paper, we propose SLIDE, the first shuffle protocol that simultaneously achieves uniformity, $O(nml)$ online communication, and guaranteed output delivery (GOD). The core of SLIDE is our novel permutation-sharing technique, with which we construct a shuffle protocol that outperforms all existing uniform shuffle constructions for Shamir secret sharing. We further integrate the shuffle-correlation technique of Gao et al. [18] to obtain linear online complexity, and finally extend the construction to GOD, completing the SLIDE framework. As shuffle is a fundamental building block for a broad class of MPC protocols and primitives, our results have wide applicability and directly benefit many practical MPC frameworks.

References

- [1] Masayuki Abe. 1999. Mix-networks on permutation networks. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 258–273.
- [2] Ben Adida and Douglas Wikström. 2007. How to shuffle in public. In *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21–24, 2007. Proceedings 4*. Springer, 555–574.
- [3] Miklós Ajtai, János Komlós, and Endre Szemerédi. 1983. An $O(n \log n)$ sorting network. In *Proceedings of the fifteenth annual ACM symposium on Theory of computing*. 1–9.
- [4] Nikolaos Alexopoulos, Aggelos Kiayias, Riivo Talviste, and Thomas Zacharias. 2017. {MCMix}: Anonymous Messaging via Secure Multiparty Computation. In *26th USENIX Security Symposium (USENIX Security 17)*. 1217–1234.
- [5] Kenneth E Batchier. 1968. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*. 307–314.
- [6] Zuzana Beerliova-Trubiniova and Martin Hirt. 2006. Efficient multi-party computation with dispute control. In *Theory of Cryptography Conference*. Springer, 305–328.
- [7] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. 2019. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*. 351–371.
- [8] Eli Ben-Sasson, Serge Fehr, and Rafail Ostrovsky. 2012. Near-Linear Unconditionally-Secure Multiparty Computation with a Dishonest Minority. In *Advances in Cryptology – CRYPTO 2012*, Reihaneh Safavi-Naini and Ran Canetti (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 663–680.
- [9] Václav E Beneš. 1964. Permutation groups, complexes, and rearrangeable connecting networks. *Bell System Technical Journal* 43, 4 (1964), 1619–1640.
- [10] Dan Bogdanov, Sven Laur, and Riivo Talviste. 2014. A practical analysis of oblivious sorting algorithms for secure multi-party computation. In *Nordic Conference on Secure IT Systems*. Springer, 59–74.
- [11] Ran Canetti. 2000. Security and Composition of Multiparty Cryptographic Protocols. *J. Cryptol.* 13, 1 (Jan. 2000), 143–202. doi:10.1007/s001459910006
- [12] Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. 2020. Secret-shared shuffle. In *Advances in Cryptology–ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part III 26*. Springer, 342–372.
- [13] David Chaum, Claude Crépeau, and Ivan Damgård. 1988. Multiparty unconditionally secure protocols. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*. 11–19.
- [14] David L Chaum. 1981. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM* 24, 2 (1981), 84–90.
- [15] Kai Chen, Bo Pang, Yongqiang Li, and Mingsheng Wang. 2025. Secure multi-party shuffling with optimal communication. *Comput. J.* (2025), bxaf043.
- [16] Saba Eskandarian and Dan Boneh. 2021. Clarion: Anonymous communication from multiparty shuffling protocols. *Network and Distributed System Security Symposium* (2021).
- [17] Brett Falk, Daniel Noble, Rafail Ostrovsky, Matan Shtepel, and Jacob Zhang. 2023. DORAM revisited: maliciously secure RAM-MPC with logarithmic overhead. In *Theory of Cryptography Conference*. Springer, 441–470.
- [18] Jiacheng Gao, Yuan Zhang, and Sheng Zhong. 2024. Multiparty Shuffle: Linear Online Phase is Almost for Free. *Cryptology ePrint Archive*, Paper 2024/1936. <https://eprint.iacr.org/2024/1936>
- [19] Vipul Goyal, Yifan Song, and Chenzhi Zhu. 2020. Guaranteed output delivery comes free in honest majority MPC. In *Annual International Cryptology Conference*. Springer, 618–646.
- [20] Jens Groth. 2010. A verifiable secret shuffle of homomorphic encryptions. *Journal of Cryptology* 23 (2010), 546–579.
- [21] Koki Hamada, Dai Ikarashi, Koji Chida, and Katsumi Takahashi. 2014. Oblivious radix sort: An efficient sorting algorithm for practical secure multi-party computation. *Cryptology ePrint Archive* (2014).
- [22] Koki Hamada, Ryo Kikuchi, Dai Ikarashi, Koji Chida, and Katsumi Takahashi. 2013. Practically efficient multi-party sorting protocols from comparison sort algorithms. In *Information Security and Cryptology–ICISC 2012: 15th International Conference, Seoul, Korea, November 28–30, 2012, Revised Selected Papers 15*. Springer, 202–216.
- [23] Feng Han, Xiao Lan, Weiran Liu, Lei Zhang, Hao Ren, Lin Qu, and Yuan Hong. 2025. Concretely Efficient Correlated Oblivious Permutation. *Cryptology ePrint Archive* (2025).
- [24] Susan Hohenberger, Guy N Rothblum, Abhi Shelat, and Vinod Vaikuntanathan. 2007. Securely obfuscating re-encryption. In *Theory of Cryptography Conference*. Springer, 233–252.
- [25] Wang Jingzhong, Zhang Yue, and Li Haibin. 2020. Electronic voting protocol based on ring signature and secure multi-party computing. In *2020 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)*. IEEE, 50–55.
- [26] Marcel Keller and Peter Scholl. 2014. Efficient, Oblivious Data Structures for MPC. In *Advances in Cryptology – ASIACRYPT 2014* (Berlin, Heidelberg, 2014), Palash Sarkar and Tetsu Iwata (Eds.). Springer, 506–525.
- [27] Ralf Küsters, Julian Liedtke, Johannes Müller, Daniel Rausch, and Andreas Vogt. 2020. Ordinos: A verifiable tally-hiding e-voting system. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 216–235.
- [28] Peeter Laud. 2021. Linear-Time Oblivious Permutations for SPDZ. In *Cryptology and Network Security*, Mauro Conti, Marc Stevens, and Stephan Krenn (Eds.). Springer International Publishing, Cham, 245–252.
- [29] Peeter Laud and Martin Pettai. 2016. Secure Multiparty Sorting Protocols with Covert Privacy. In *Secure IT Systems*, Billy Bob Brumley and Juha Röning (Eds.).

- Springer International Publishing, 216–231.
- [30] John Launchbury, Iavor S Diatchki, Thomas DuBuisson, and Andy Adams-Moran. 2012. Efficient lookup-table protocol in secure multiparty computation. In *Proceedings of the 17th ACM SIGPLAN international conference on Functional programming*. 189–200.
 - [31] Sven Laur, Jan Willemson, and Bingsheng Zhang. 2011. Round-efficient oblivious database manipulation. In *Information Security: 14th International Conference, ISC 2011, Xi'an, China, October 26-29, 2011. Proceedings 14*. Springer, 262–277.
 - [32] Tom Leighton and C Greg Plaxton. 1990. A (fairly) simple circuit that (usually) sorts. In *Proceedings [1990] 31st Annual Symposium on Foundations of Computer Science*. IEEE, 264–274.
 - [33] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. 2019. HoneyBadgerMPC and AsynchroMix: Practical Asynchronous MPC and its Application to Anonymous Communication. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (London, United Kingdom) (CCS '19)*. Association for Computing Machinery, New York, NY, USA, 887–903. <https://doi.org/10.1145/3319535.3354238>
 - [34] Dhaneshwar Mardi, Surbhi Tanwar, and Jaydeep Howlader. 2021. Multiparty protocol that usually shuffles. *Security and Privacy* 4, 6 (2021), e176.
 - [35] Mahnush Movahedi, Jared Saia, and Mahdi Zamani. 2015. Secure multi-party shuffling. In *Structural Information and Communication Complexity: 22nd International Colloquium, SIROCCO 2015, Montserrat, Spain, July 14-16, 2015. Post-Proceedings 22*. Springer, 459–473.
 - [36] Mahnush Movahedi, Jared Saia, and Mahdi Zamani. 2015. Shuffle to baffle: Towards scalable protocols for secure multi-party shuffling. In *2015 IEEE 35th International Conference on Distributed Computing Systems*. IEEE, 800–801.
 - [37] Stanislav Peceny, Srinivasan Raghuraman, Peter Rindal, and Harshal Shah. 2025. Efficient permutation correlations and batched random access for two-party computation. In *IACR International Conference on Public-Key Cryptography*. Springer, 76–109.
 - [38] Allan B Pedin IV and Nazli Siasi. 2023. Secure and Decentralized Anonymous E-Voting Scheme. In *Proceedings of the 2023 ACM Southeast Conference*. 172–176.
 - [39] Adi Shamir. 1979. How to share a secret. *Commun. ACM* 22, 11 (1979), 612–613.
 - [40] Xiangfu Song, Dong Yin, Jianli Bai, Changyu Dong, and Ee-Chien Chang. 2023. Secret-Shared Shuffle with Malicious Security. *Network and Distributed System Security Symposium* (2023).
 - [41] Chenghong Wang, Johes Bater, Kartik Nayak, and Ashwin Machanavajjhala. 2022. IncShrink: architecting efficient outsourced databases using incremental mpc and differential privacy. In *Proceedings of the 2022 International Conference on Management of Data*. 818–832.
 - [42] Yanping Zhang, Johes Bater, Kartik Nayak, and Ashwin Machanavajjhala. 2023. Longshot: Indexing growing databases using MPC and differential privacy. *Proceedings of the VLDB Endowment* 16, 8 (2023), 2005–2018.

A Security Proofs

In this section, we analyze our protocols and prove that the shuffle protocols are secure. By virtue of the ideal functionality F_{Shamir} and the work of Gao et al. [18], the security follows almost immediately from the definitions. Nevertheless, we provide in this section some insights into the security of our protocols.

To simplify our analysis, we focus on detecting the adversary’s misbehavior. That is, we prove that if the adversary deviates from the protocol, the protocol will abort with overwhelming probability. This approach is justified by noting that in standard simulation-based security, one must construct a simulator that can simulate the adversary’s view while being ignorant of the honest parties’ inputs. However, this simulation is almost always trivial in Shamir secret sharing: the simulator represents all honest parties, enabling it to reconstruct all values shared by the adversary. Hence, in a typical proof, the simulator simply sends random values to the adversary as the messages it receives, aborts if any check fails, and finally modifies the shares for honest parties to align them with the output of the ideal functionality. This means we only need to argue that any misbehavior will lead to abort with overwhelming probability, ensuring the simulator’s output coincides with the real distribution.

Note that all basic operations in our construction rely on ideal functionalities, which provide (seemingly extremely) strong security guarantees—e.g., even introducing an error in Add will trigger

an immediate abort. This strong condition is justified by the composition theorem, which states that a protocol remains secure after replacing ideal functionalities with their secure implementations. Thus, any misbehavior against ideal functionalities will be detected, and we only need to prove that misbehavior outside the ideal functionalities will be detected.

Hence, we focus on showing that the protocol ends in one of two cases: (1) the adversary misbehaves, and the protocol aborts with overwhelming probability; (2) all parties act honestly, and honest parties receive correct shares.

As shown in Gao et al. [18], if the Permute protocol is secure, then $\text{Shuffle}_{2\text{off}}$ and $\text{Shuffle}_{2\text{on}}$ are also secure. This is formalized in Theorem A.1 below.

THEOREM A.1 ([18], INFORMAL). *Consider the ideal functionality*

$$\llbracket \pi(\mathbf{v}) \rrbracket \leftarrow \Pi_{\text{perm}}(P_w : \pi, \llbracket \mathbf{v} \rrbracket),$$

which takes a secret input π from P_w and securely permutes $\llbracket \mathbf{v} \rrbracket$ to $\llbracket \pi(\mathbf{v}) \rrbracket$.

If Permute securely implements Π_{perm} , then $\text{Shuffle}_{2\text{off}}$ and $\text{Shuffle}_{2\text{on}}$ are secure by the composition theorem. This means:

- *If corrupted parties misbehave, all parties abort with overwhelming probability;*
- *If the protocol does not abort, $\mathbf{v}$ is correctly shuffled and shared among honest parties with overwhelming probability, and the adversary learns no information about the applied permutation.*

This theorem allows us to focus on the security of the Permute protocol. Since Permute only invokes basic primitives and previously developed protocols, it suffices to prove the security of its sub-protocols.

A.1 Security of One-hot Protocol

The security of the one-hot vector sharing protocol follows almost directly from the basic primitives, which we briefly review here.

LEMMA A.2. *The protocol $\text{BinCheck}(\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket)$ described in Algorithm 2 is secure: if any input is non-binary (i.e., not in $\{0, 1\}$), it aborts with probability*

$$p \geq 1 - \frac{d-1}{2^\kappa}.$$

Proof: Since the computation uses the ideal functionalities Π_{open} and Π_{inner} , the parties correctly receive the opened value:

$$u = \sum_{i=1}^d b_i \cdot (1 - b_i) \cdot \lambda^{i-1}.$$

λ is uniformly random over $\mathbb{K}$ (generated by $\Pi_{\text{challenge}}$). In $\mathbb{K}$, the product $b_i(1 - b_i)$ is zero if and only if $b_i \in \{0, 1\}$. If any b_i is non-binary, the polynomial $F(X) = \sum_{i=1}^d b_i(1 - b_i)X^{i-1}$ is non-zero and of degree at most $d - 1$. A non-zero polynomial of degree $d - 1$ has at most $d - 1$ roots in $\mathbb{K}$, so the probability that λ is a root is:

$$p_{\text{fail}} \leq \frac{d-1}{|\mathbb{K}|} \leq \frac{d-1}{2^\kappa}.$$

Hence, $u \neq 0$ (and the protocol aborts) with probability:

$$p = 1 - p_{\text{fail}} \geq 1 - \frac{d-1}{2^\kappa}.$$

LEMMA A.3. *If all inputs are shared bits, the protocol $\text{Demux}(\llbracket b_1 \rrbracket, \dots, \llbracket b_d \rrbracket)$ described in Algorithm 3 is secure: if the protocol does not abort, honest parties share $\llbracket s \rrbracket$ as a one-hot vector with index $1 + \sum_{i=1}^d 2^{i-1} b_i$.*

Proof: For the base case $d = 1$, honest parties hold:

$$\llbracket s \rrbracket = (\llbracket 1 - b_1 \rrbracket, \llbracket b_1 \rrbracket),$$

which is correct and secure.

For $d > 1$, the protocol essentially computes the tensor product of two one-hot vectors via calls to the ideal functionality Π_{mul} . Correctness and security follow trivially.

THEOREM A.4. *The protocol $\text{OneHot}(k, P_w : \text{ind})$ described in Algorithm 4 is secure: upon normal exit, honest parties hold a valid one-hot vector with index chosen by P_w .*

Additionally, if P_w is honest, corrupted parties learn no information about the chosen index.

Proof: If P_w is honest, the result is a one-hot vector with the chosen index upon normal exit. Further, BinCheck opens a zero value, which only reveals that the inputs are binary (trivial information).

If P_w is malicious and shares $\llbracket b'_i \rrbracket$ during the OneHot protocol:

- If any $b'_i \notin \{0, 1\}$, BinCheck aborts with overwhelming probability (by Lemma A.2);
- If all $b'_i \in \{0, 1\}$, there exists a valid index $\text{ind}' = \sum_{i=1}^d 2^{i-1} b'_i + 1$ for the generated vector. Since P_w may freely choose the index, this does not compromise security.

A.2 Security of Permutation Protocol

THEOREM A.5. *If the input k one-hot vectors do not form a valid $k \times k$ permutation matrix, the protocol $\text{PermCheck}(\llbracket P \rrbracket)$ described in Algorithm 6 aborts with probability*

$$p \geq 1 - \frac{k-1}{2^\kappa}.$$

Proof: The parties compute:

$$\llbracket \text{sum} \rrbracket \leftarrow \sum_{j=1}^k \lambda^{j-1} \left(1 - \sum_{i=1}^k \llbracket P(i, j) \rrbracket \right).$$

Since these arithmetic operations use ideal functionalities, the parties receive the correct $\llbracket \text{sum} \rrbracket$ (unless the protocol already aborted due to misbehavior).

- If P is a valid permutation matrix, $\sum_{i=1}^k P(i, j) = 1$ for all j , so $\llbracket \text{sum} \rrbracket = \llbracket 0 \rrbracket$;
- If P is invalid, at least one coefficient $1 - \sum_{i=1}^k P(i, j)$ is non-zero, making $\llbracket \text{sum} \rrbracket$ a non-zero polynomial of λ with degree at most $k-1$.

A non-zero polynomial of degree $k-1$ has at most $k-1$ roots in $\mathbb{K}$. Since λ is uniform over $\mathbb{K}$, the probability that λ is a root is:

$$p_{\text{fail}} \leq \frac{k-1}{|\mathbb{K}|} \leq \frac{k-1}{2^\kappa}.$$

Thus, $\llbracket \text{sum} \rrbracket \neq \llbracket 0 \rrbracket$ (and the protocol aborts) with probability:

$$p = 1 - p_{\text{fail}} \geq 1 - \frac{k-1}{2^\kappa}.$$

COROLLARY A.6. *The protocol PermMat described in Algorithm 5 is secure:*

- *If all parties act honestly, honest parties share a valid permutation matrix $\llbracket P \rrbracket$ with permutation chosen by and known to P_w ;*
- *If the malicious adversary misbehaves, the protocol aborts with overwhelming probability.*

Additionally, if P_w is honest, corrupted parties learn no information about P .

Proof: Immediate from Theorem A.4 and Theorem A.5.

COROLLARY A.7. *The protocol SharePerm described in Algorithm 7 is secure:*

- *Any adversary misbehavior leads to abort with overwhelming probability;*
- *If all parties act honestly, honest parties share $\llbracket \pi \rrbracket$ with π chosen by and known to P_w .*

Additionally, if P_w is honest, the adversary learns no information about π .

Proof: By Corollary A.6, if the protocol has not aborted, all shared matrices are valid permutation matrices—implying the permutation is correctly shared.

COROLLARY A.8. *The protocol $\text{Permute}(\llbracket \pi \rrbracket, \llbracket v \rrbracket)$ described in Algorithm 1 is secure:*

- *Any adversary misbehavior leads to abort with overwhelming probability;*
- *If all parties act honestly, honest parties share $\llbracket \pi(v) \rrbracket$.*

This follows trivially from the definition of shared permutation and the security of ideal functionalities.

This concludes the security proof of our permutation protocol.

A.3 Security of Shuffle Protocol

Finally, we conclude the security of the shuffle protocols by combining Theorem A.1, Corollary A.7, and Corollary A.8.

COROLLARY A.9. *The $\text{Shuffle1}_{\text{off}}$ and $\text{Shuffle1}_{\text{on}}$ protocols described in Algorithms 8 and 9 are secure:*

- *They abort with overwhelming probability if the adversary misbehaves;*
- *Otherwise, they shuffle $\llbracket v \rrbracket$ using a secret permutation uniformly random over all m -permutations.*

Proof: Direct result from Corollary A.7 and Corollary A.8.

COROLLARY A.10. *The $\text{Shuffle2}_{\text{off}}$ and $\text{Shuffle2}_{\text{on}}$ protocols described in Algorithms 11 and 12 are secure:*

- *They abort with overwhelming probability if the adversary misbehaves;*
- *Otherwise, they shuffle $\llbracket v \rrbracket$ using a secret permutation uniformly random over all m -permutations.*

Proof: Direct result from Theorem A.1, Corollary A.7, and Corollary A.8.

B Proofs of Complexity Analysis

B.1 Proofs for Section 4.6

THEOREM B.1. *For protocol $\text{Permute}(\llbracket \pi \rrbracket_k, \llbracket v \rrbracket)$ described in Algorithm 1, the communication, computation and round complexity is $O(\frac{1}{\log k} nm \log m)$, $O(\frac{k}{\log k} nm \log m)$ and $O(\frac{\log m}{\log k})$, respectively.*

Proof: The parties are computing $O(\frac{m \log m}{k \log k})$ many matrix-vector products, each multiplying a $k \times k$ permutation matrix by a k -long data vector. Such a multiplication is simply k calls to Π_{inner} , each of which has a communication complexity of only $O(nk)$. The communication complexity is hence $O(\frac{1}{\log k} nm \log m)$. The computation complexity of a single matrix-vector product is $O(nk^2)$, and the computation complexity is hence $O(\frac{k}{\log k} nm \log m)$.

For round complexity, note that the parties can parallelize all small permutations $\pi'_{i,j}$ with the same i . The entire i -th iteration is thus done within $O(1)$ rounds, and the round complexity is simply $O(s) = O(\frac{\log m}{\log k})$.

LEMMA B.2. For protocol $\text{BinCheck}([b_1, \dots, b_d])$ described in Algorithm 2, the communication, computation and round complexity is $O(n)$, $O(nd)$ and $O(1)$, respectively.

Proof: The only communication occurs via one call to each of $\Pi_{\text{challenge}}$, Π_{open} , and Π_{inner} —all of which incur $O(n)$ communication and $O(1)$ rounds.

The computation bottleneck is computing

$$[u] \leftarrow \sum_{i=1}^d \lambda^{i-1} [b_i] [1 - b_i],$$

which totals $O(nd)$ computation.

LEMMA B.3. For protocol $\text{Demux}([b_1, \dots, b_d])$ described in Algorithm 3, the communication, computation and round complexity is $O(n2^d)$, $O(n2^d)$ and $O(\log d)$, respectively.

Proof: Denote by $M(d)$ the number of calls to the primitive Π_{mul} . Then

$$M(d) = 2 \times M(d/2) + 2^d.$$

Since for all $d \geq 4$,

$$2^{d+1} \geq 2 \times 2^{d/2+1} + 2^d = 2^{d/2+2} + 2^d,$$

we conclude that $M(d) = O(2^{d+1}) = O(2^d)$.

Recall that in protocol Demux , only calls to Π_{mul} require communication. Hence, the communication complexity is $O(n2^d)$. The computation complexity follows similarly.

Denote by $R(d)$ the number of rounds. Note that

$$R(d) = R(d/2) + O(1).$$

This is because the two calls to Demux can be parallelized, and the remainder are parallel calls to Π_{mul} . Hence, the round complexity is $O(\log d)$.

LEMMA B.4. For protocol $\text{OneHot}(k, P_w : \text{ind})$ described in Algorithm 4, the communication, computation and round complexity is $O(nk)$, $O(nk)$ and $O(\log \log k)$, respectively.

Proof: The bottleneck is the call to $\text{Demux}([b_1, \dots, b_d])$, for $d = \log k$. According to Lemma B.3, its communication, computation and round complexity is $O(n2^d)$, $O(n2^d)$ and $O(\log d)$, respectively. As $d = \log k$, the lemma follows trivially.

LEMMA B.5. For protocol $\text{PermCheck}([P])$ described in Algorithm 6, the communication, computation and round complexity is $O(n)$, $O(nk^2)$ and $O(1)$, respectively.

Proof: The communication occurs only at $\Pi_{\text{challenge}}$ and Π_{open} , each of which costs $O(n)$ communication and $O(1)$ rounds.

The most computation-heavy step in the algorithm is each party locally computing the sums, which costs $O(nk^2)$ in total.

LEMMA B.6. For protocol $\text{PermMat}(P_w : \pi)$ described in Algorithm 5, the communication, computation and round complexity is $O(nk^2)$, $O(nk^2)$ and $O(\log \log k)$, respectively.

Proof: The protocol consists of k parallel calls to protocol $\text{OneHot}(k, P_w : \pi(i))$ plus one call to $\text{PermCheck}([P])$. By Lemma B.4 and Lemma B.5, this costs $O(nk^2)$ communication, $O(nk^2)$ computation and $O(\log \log k)$ rounds.

THEOREM 4.3. For protocol $\text{SharePerm}(k, P_w : \pi)$ given in Algorithm 7, the communication, computation and round complexity is $O(\frac{k}{\log k} nm \log m)$, $O(\frac{k}{\log k} nm \log m)$ and $O(\log \log k)$, respectively.

Proof: Recall that the permutation decomposition of [12] decomposes an m -permutation π into $s = O(\frac{\log m}{\log k})$ many m -permutations $\pi_1, \dots, \pi_s$. Each permutation π_i is then decomposed into m/k many permutations $\pi_{i,1}, \dots, \pi_{i,m/k}$, each transformed into a k -permutation $\pi'_{i,j}$.

To share the permutations, P_w needs to share $O(\frac{m \log m}{k \log k})$ many $k \times k$ permutation matrices in total. According to Lemma B.6, parallel calls to PermMat result in $O(\frac{k}{\log k} nm \log m)$ communication and computation, and $O(\log \log k)$ rounds.

THEOREM 4.4. Consider an input collection $[V] = [v_1, \dots, v_m]$ of m vectors, each of length l , to be jointly shuffled by the parties. For our first shuffle protocol described in Algorithm 8 and 9, the offline communication, computation and round complexity is $O(\frac{k}{\log k} n^2 m \log m)$, $O(\frac{k}{\log k} n^2 m \log m)$ and $O(\log \log k)$, respectively. The online communication, computation and round complexity is $O(\frac{l}{\log k} n^2 m \log m)$, $O(\frac{kl}{\log k} n^2 m \log m)$ and $O(\frac{n \log m}{\log k})$, respectively.

Proof: Note that all parties can share permutation matrices in parallel in the offline phase. Hence, the offline round complexity is simply $O(\log \log k)$.

Other complexities follow naturally from Theorem B.1 and the theorem for SharePerm (Theorem 4.3).

B.2 Proofs for Section 5.4

THEOREM 5.2. For $\text{Shuffle2}_{\text{off}}(P_1 : \pi_1, \dots, P_n : \pi_n)$ described in Algorithm 11, the (offline) communication, computation and round complexity is $O(\frac{k}{\log k} n^2 m \log m)$, $O(\frac{k}{\log k} n^2 m \log m)$ and $O(\frac{\log m}{\log k} + \log \log k)$, respectively.

Proof: There are in total $n \cdot m$ calls to each of Π_{rand} , Π_{mul} and Π_{open} . This requires $O(n^2 m)$ communication, which is not the bottleneck.

The bottleneck of all the complexities is the n calls to protocols SharePerm and Permute , each for permuting an $m \times 3$ matrix. Since these calls are in parallel, by combining the offline and online complexities presented in Theorem B.1 and the theorem for SharePerm (Theorem 4.3), we obtain that the communication, computation and round complexity are $O(\frac{k}{\log k} n^2 m \log m)$, $O(\frac{k}{\log k} n^2 m \log m)$ and $O(\frac{\log m}{\log k} + \log \log k)$, respectively.

THEOREM 5.3. *For $\text{Shuffle}_{2\text{on}}(\llbracket \mathbf{v} \rrbracket)$ described in Algorithm 12, the communication, computation and round complexity is $O(nm + n^2)$, $O(n^2m)$ and $O(n)$, respectively. Note that for $n = O(m)$, the communication is just $O(nm)$.*

Proof: Note that protocol CorrCheck , when called for a vector of length m , costs $O(n)$ communication, $O(nm)$ computation and $O(1)$ rounds. Besides calls to CorrCheck , parties compute Π_{mul} over an m -long vector, and each party P_i sends $\mathbf{y}_i$ of length $O(m)$ to P_{i+1} . P_n lastly broadcasts $\mathbf{y}_n$, which requires $O(nm)$ communication.

Summing all up, the communication complexity is $O(n(n + m))$, the computation complexity is $O(n^2m)$ due to n calls to CorrCheck , and the round complexity is $O(n)$.

C Discussion

C.1 Sharing Permutation Matrix

Recall that the parties wish to perform computation in some field $\mathbb{F}$ with prime subfield $\mathbb{F}_p$. Suppose party P_w needs to share an $m \times m$ permutation matrix. When p is larger than m , generating and checking a permutation matrix can be much easier.

To generate a (purported) permutation matrix chosen by party P_w , the protocol simply requires P_w to share the matrix. This matrix will be a valid permutation matrix if the following conditions are satisfied:

- (1) Each entry of the matrix is either 0 or 1 (binary property).
- (2) The sum of entries along each row is exactly 1.
- (3) The sum of entries along each column is exactly 1.

To see why this holds, note that the first condition ensures the matrix is binary. Since $p > m$, the sum along any row or column does not suffer from modular overflow—meaning the sum directly equals the number of 1s in that row or column. Hence, a binary matrix with exactly one 1 in each row and each column is by definition a permutation matrix. Notably, the first condition can be verified with a single invocation of Π_{inner} , and all three conditions can be batched into a single invocation of Π_{open} . Thus, this check incurs almost the same overhead as the PermCheck protocol.

However, the above argument is valid only when $p > m$. A counterexample is when $m = p + 1$: a malicious party could share an all-ones matrix and pass all the checks, since $p + 1 \equiv 1 \pmod{p}$ in such a field (the sum of $p + 1$ ones modulo p equals 1). In the case of $p \leq m$, it is significantly harder to check whether an arbitrary matrix is a valid permutation matrix—modular overflow makes row/column sum checks unreliable.

Hence, in our construction, we impose specific constraints (via one-hot vectors) to simplify such checks in the general case (for any p and m). The asymptotic communication complexities of the two approaches (direct check for $p > m$ vs. one-hot vector-based check for general p) are identical: both are $O(nm^2)$ per permutation matrix (for generation and verification).

C.2 Online Complexity

There are several reasons why the online complexity of an MPC protocol is far more important than its offline complexity. First, if we construct an anonymous communication system based on an MPC shuffle (e.g., Clarion by [16]), the online complexity corresponds to the latency experienced by users or clients. Similar reasoning

applies to other real-world MPC applications (e.g., private data analysis, secure multi-party computation for financial transactions).

Another profound reason is that the offline phases of multiple protocol sessions can be executed in parallel. This allows for a significant reduction in the protocol’s round complexity. For example, suppose parties aim to perform shuffle sequentially k times, where each execution requires R_{off} rounds in the offline phase and R_{on} rounds in the online phase. Through naive parallelization of data-independent operations, the total computation requires only R_{off} offline rounds (for all k sessions) and $k \cdot R_{\text{on}}$ online rounds (one after another for each session). This effect is even more pronounced if the shuffle protocol serves as a subroutine of a higher-level protocol.

A concrete example is the MPC radix sort algorithm by Hamada et al. [21], which relies on MPC shuffle as a core primitive. To sort data by 128-bit indices, 128 sequential invocations of the shuffle protocol are required—this totals at least $R_{\text{off}} + 128 \times R_{\text{on}}$ rounds. Since only the online phase is inevitably embedded as a subroutine into higher-level protocols, only the online round complexity is subject to this kind of amplification. This renders the online round complexity far more important than the offline counterpart.

C.3 Definition of Shared Permutation

The requirement that $s = O\left(\frac{\log m}{\log k}\right)$ in Definition 4.1 is tight (i.e., the bound cannot be improved). To see this, note that there are $m!$ distinct m -permutations. The number of possible k -shared m -permutations (as defined earlier) is at most $(k!)^{ms/k}$ —this number must be at least $m!$ to cover all m -permutations. Hence:

$$s \geq \log_{(k!)^{m/k}}(m!) = \frac{k \log(m!)}{m \log(k!)}.$$

By Stirling’s approximation, $\log(m!) = \Theta(m \log m)$ and $\log(k!) = \Theta(k \log k)$. Substituting these into the inequality gives $s = \Omega\left(\frac{\log m}{\log k}\right)$, which matches the upper bound $s = O\left(\frac{\log m}{\log k}\right)$ —proving tightness.

Note that one may also consider a recursive definition, where $\llbracket \mathbf{P}_{i,j} \rrbracket$ is replaced by a recursive $\llbracket \pi_{i,j} \rrbracket$. The recursion terminates when a given $\llbracket \pi \rrbracket$ is shared exactly as $\llbracket \mathbf{P}_\pi \rrbracket$ (a direct permutation matrix share). Nevertheless, the current definition is sufficient for our purposes—recursion would add unnecessary complexity without improving performance.